

4IE2

情報エレクトロニクス実験Ⅲ

SQL データベース

目次

1. 概要	5
2. データベースの概要	6
2.1 データベースとは	6
2.2 データベースマネージメントシステム	6
2.3 データベースの種類	7
2.4 リレーショナルデータベース	9
(1) 基本機能	9
(2) 信頼性のための機能	9
(3) SQL	9
(4) MySQL	10
2.5 データベースの構造	10
(1) データベース	10
(2) テーブル	11
3. データベースの構築	14
3.1 データの関係を考える	14
3.2 データのコード化	14
3.3 テーブルと関連の図式化	15
(1) 論理モデル	15
(2) 物理モデル	15
(3) 構成要素	16
3.4 テーブル設計	17
(1) 顧客台帳(customer)	17
(2) 商品台帳(goods)	17
(3) 売上台帳(sales)	17
3.5 データベース作成と接続	18
3.6 テーブル作成	18
(2) データ入力	19
4. ER 図の作成	21
4.1 astah を立ち上げる	21
4.2 ER 図を新規作成する	21
4.3 エンティティ	22
(1) エンティティを追加する	22
(2) 追加されたエンティティ	22
(3) 主キーを追加する	22
(4) 属性を追加する	23
(5) 他のテーブル, 論理名を追加する	23
4.4 リレーション	24
(1) リレーションの接続元	24

(2) 接続先	24
4.5 物理名	26
(1) テーブル名	26
(2) 主キー名	26
(3) 属性名	27
(4) 物理モデルを表示する	27
4.6 SQL 出力	28
(1) SQL エクスポート	28
(2) 「オプション」を選択する	28
(3) 「CREATE TABLE 文のみ使用する」にチェック	29
(4) 出力する	29
5. データベースの設計	30
(1) フィールドの洗い出し	30
(2) 第 1 正規化	30
(3) 第 2 正規化	30
(4) 第 3 正規化	33
(5) テーブル実装	34
6. SQL	37
6.1 データベース作成	37
6.2 テーブル作成	38
6.3 挿入	39
6.4 選択	40
(1) そのまま表示	40
(2) 条件	42
(3) 機能	44
6.5 削除	45
(1) データベース削除	45
(2) テーブル削除	45
(3) データ削除	45
6.6 更新	45
6.7 設定変更	46
(1) データ型変更	46
(2) カラム追加	47
(3) カラム名	47
(4) カラム削除	48
(5) 主キー設定	48
(6) 主キー削除	49
6.8 JOIN	50
(1) 内部結合 (INNER JOIN)	52
(2) 外部結合 (OUTER JOIN)	53

7. ファイル	54
7.1 ディレクトリ設定	54
7.2 ファイルからの INSERT	55
(1) タブ区切り	55
(2) CSV	56
7.3 コマンド読み込み	57
7.4 Excel のデータ出力	58
8. インストール	60
8.1 ダウンロード	60
(1) 以下のアドレスをウェブブラウザで開く	60
(2) 「ダウンロード」メニューを選ぶ	60
(3) 「MySQL Community Server」をクリックする	61
(4) Windows(x86, 64bit), MSI Installer 128.4M の方をダウンロード	61
(5) No thanks, just start my download	62
8.2 インストール開始	63
(1) 「I accept the terms in the License Agreement」にチェックを入れる	63
(2) 「Typical」を選択	64
(3) 「Install」をクリック	64
(4) 「MySQL Configurator」が立ち上がる	65
(5) ディレクトリの指定	66
(6) 「Open Windows Firewall ports for network access」のチェックを外す	66
(7) パスワードを決める	67
(8) 「Execute」をクリックしてしばらく待つ	69
(9) 「Finish」をクリックしてインストール完了	70
8.3 設定	71
(1) 設定ファイルを開く	71
(2) 設定する	72
8.4 実行	73
(1) スタートメニューから「MySQL」と入力する	73
(2) パスワードを入力する	73
(3) SQL のコマンドラインツールが立ち上がる	73
9. 課題	74
9.1 課題 1	74
9.2 課題 2	75
9.3 課題 3	76
10. 提出	77
レポート本体 (PDF ファイル)	78
参考文献	78

1. 概要

多くのデータを扱いたいとき、ファイルにデータを保存して、そのファイルを読み込んで多くのデータを取り扱うことがある。しかし、ファイルを使うと、複数の人が同時に編集することが難しくなる。さらに、大量のデータから検索したり、更新したりする処理を扱うことが難しくなることがある。

そこで、データベースを使ってデータを保存する、抽出する、整理する方法を学ぶ。データベースとやり取りするための言語として、SQL がある。SQL を使って、データベースを使う方法を学ぶ。

また、データベースにデータを効率よく格納して、拡張性を考慮した、テーブルの定義の仕方として正規化がある。正規化の種類を把握し、実際にテーブルを定義する際に適用する。また、その正規化したテーブルを用いて、所望のデータを構成するための SQL の命令を使って、データベースから任意のデータを抽出する方法を学ぶ。

2. データベースの概要

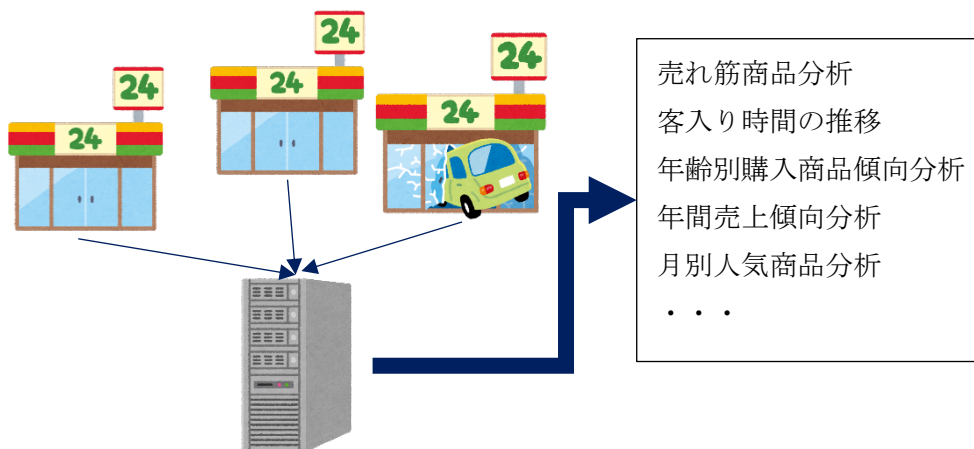
2.1 データベースとは

データベースとは、一般に「データが整理されて保管されているもの」という意味がある。例えば、図1のラーメンデータベースなど、ラーメン屋さんの情報が集められているものをデータベースと呼ぶこともあるだろう。



図1 ラーメンデータベースの例

他方、「データを活用する」ことを主眼として、蓄積されたデータを分析したり、予測に利用するために、たくさんのデータを格納しておくことができ、抽出したり、分析できるようにするための「システム」としてのデータベースがある。情報システムの一部としてのデータベースは、データを保存する重要な役割をもっている。



2.2 データベースマネージメントシステム

コンピュータシステムにおける「データベース」というと、データベースマネージメントシステム (DBMS) を指すことが多い。データベースマネージメントシステム (DBMS) は、格納

されているデータ群から任意の条件に当てはまるものを抽出したり，順番を並べかえたりすることで，必要なデータにアクセスできるようにし，アプリケーションなどから利用できるように，システムの一部をなす。

データベースは，データベースマネジメントシステムからデータにアクセスしやすい形式にまとめられていて，ファイルになって保存されている場合が多い。その中から，必要なデータを取り出して提供するのが，DBMS の役割である。データを取り扱うアプリケーションは，DBMS を介して，データにアクセスすることになる。データベースを利用するシステムの構造の概念図を図 2 に示す。

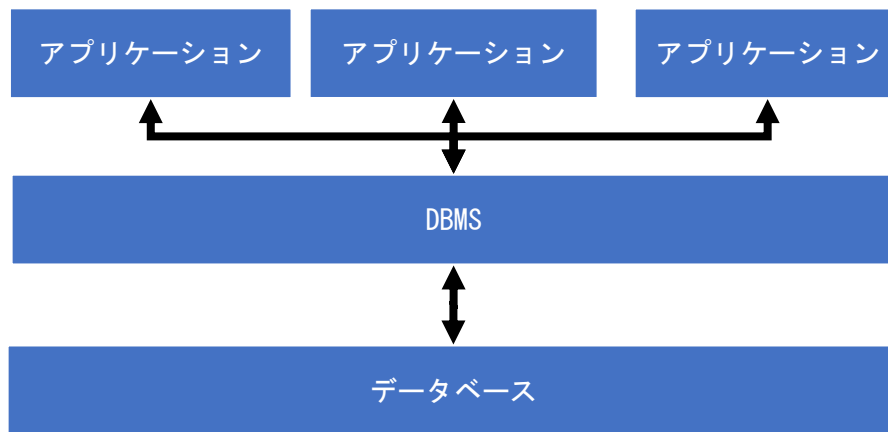


図 2 データベースを利用するシステムの構造

データベースを利用するシステムと，図書の検索との比較イメージを図 3 に示す。

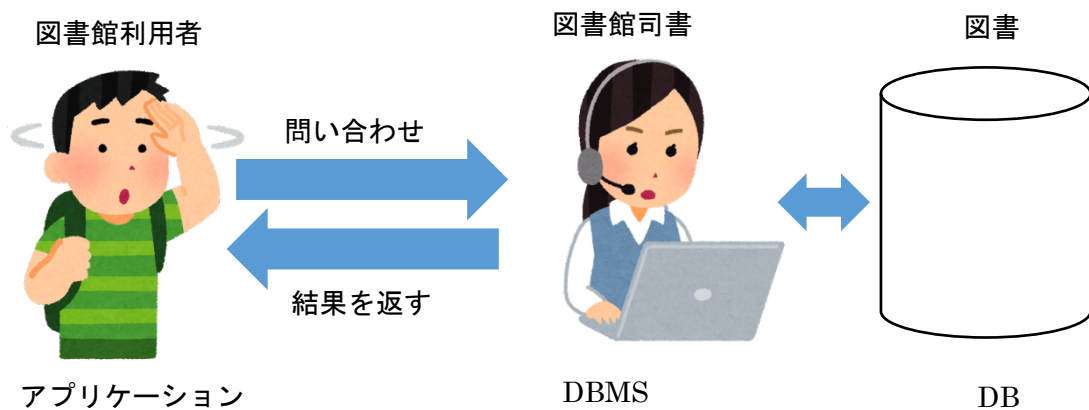


図 3 データベースを利用するシステムのイメージ

DBMS には，プロプライエタリなデータベースとして，オラクル社の Oracle Database, IBM 社の DB2, オープンソースなデータベースとして，MySQL, PostgreSQL などがある。

データベースは，データを扱うことから，非常に堅牢であることが求められる。例えば，銀行のシステムでは，預金のデータに間違いや不整合があれば，預金者の資産に影響をおよぼすことや，商取引に影響を与えることがある。そのため，データベースには高い信頼性が求められる。

2.3 データベースの種類

データベースには，データの格納方法などによりいくつか種類がある。

図 4 のように，ネットワーク型データモデル，階層型データモデル，カード型データモデル，関係データモデルなどがある。これらのなかで，昨今広く使われているデータベースのモデルは

図 5 のような、関係データモデル（リレーショナルデータベース）である。

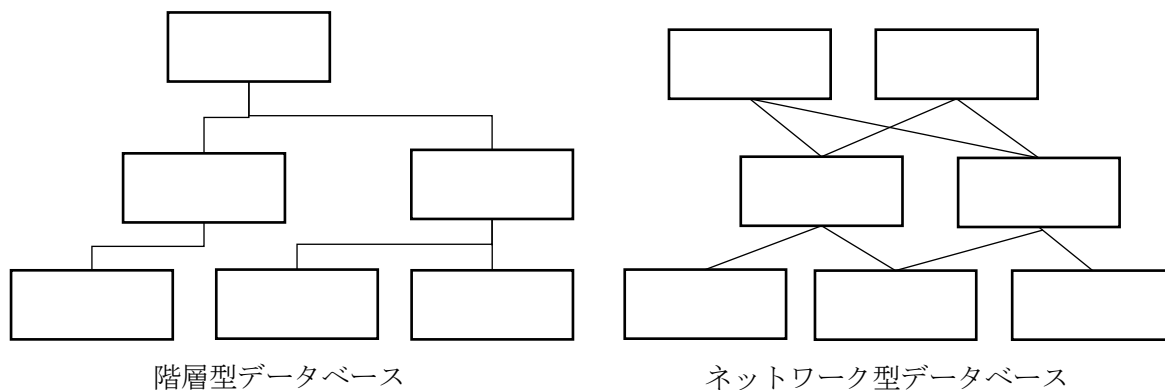


図 4 様々なデータベースの形式 (1)

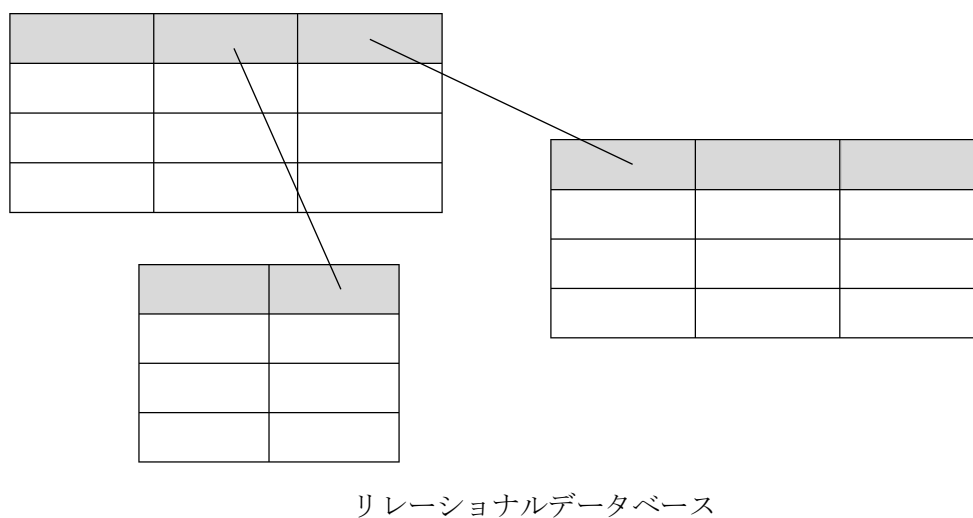


図 5 様々なデータベースの形式 (2)

最近では、IoT をコンセプトに、データの新しい活用の仕方が話題になっている。その様な場合には、ドキュメント型（図 6）など、リレーショナルでないデータベースをさす「NoSQL」と呼ばれることがある。

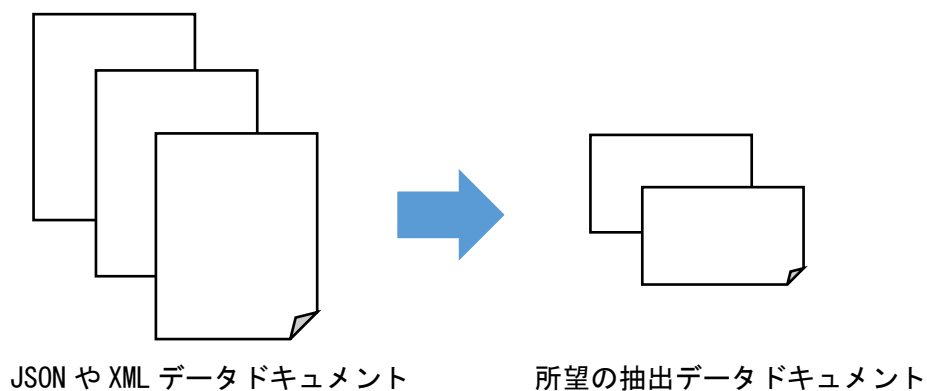


図 6 ドキュメント型データベースのイメージ

ドキュメント型データベースでは、データ形式が多様で、入れ替わりが頻繁なデータを取り扱うために、データベースの定義を厳密にせず、格納したデータベースからデータを抽出でき

るようにして、柔軟なデータ処理ができるように考えられている。

2.4 リレーショナルデータベース

表のように、カラムが設定され、レコードが記録されたテーブルが、相互に関係（リレーション）を取ることで、所望のデータを抽出することができるような仕組みを持ったデータベースである。

(1) 基本機能

ここで求められる機能として、以下のようなものがある。

(a) 格納

たくさんのデータを格納できる。

例) 1 日 100 件の注文がある。100 万件の発注データを格納する。

(b) 検索

多くのデータの中から目的のデータを検索できる。

例) 格納されている発注 100 万件のデータから 500m 秒内に検索結果を返す。3 分以内に、月別の売上を集計できる。

(c) 操作

データを追加，削除，更新できる。

例) データを 1 分に 10 件追加することができ，削除ができる。指定したデータを呼び出して，更新することができる。

(d) 定義

データ構造を定義して，それに関係づけができる。

(2) 信頼性のための機能

そのほか，今回の授業では，取り扱わないが，データベースの基本的な機能として，以下のようなものがある

(a) 同時制御

複数のユーザが同時にアクセスしたときに，データの不整合を起こさないように，タイミングや，範囲を制御する。

(b) トランザクション処理

データの変更などが，複数の処理にわたるときに，不整合が生じないように完全に行うか，途中経過も含めてもとに戻すようにするなどにより，データベースの一貫性を保つための処理である。

(c) 障害回復

障害が発生したときに，回復できるように，バックアップ・リストアできるようにしたり，ログファイルから障害が発生する処理の前まで戻したり（ロールバック），処理前から処理後の状態に進めたり（ロールフォワード）して，回復できるように，耐久性を保つための機能である。

(3) SQL

多くの場合，RDB への問い合わせとして，SQL（Structured Query Language）が用いられる。SQL は，カラムが定義されたテーブルが関係づけされたデータに対して，所望のデータを取出すために問い合わせ（Query）をするための言語である。図 7 のように，問い合わせしたいことを DBMS を介して，DB からデータを抽出する。

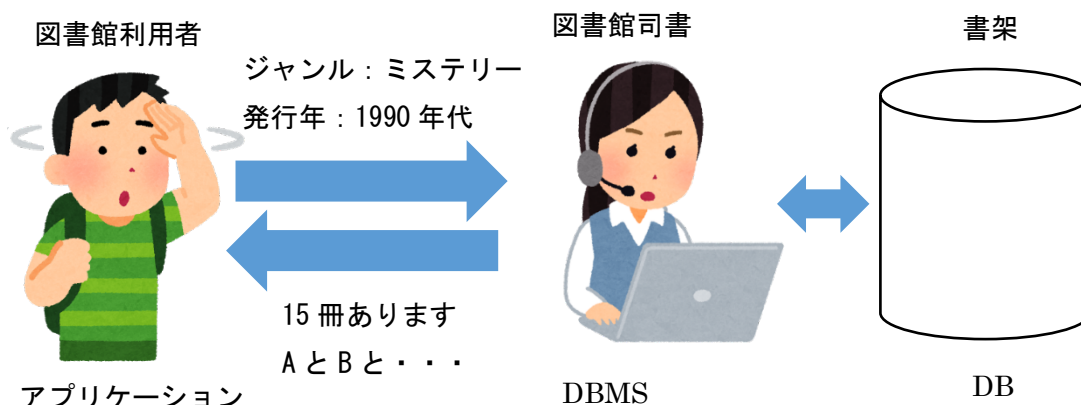


図 7 SQL を用いた DBMS を通した流れ

「言語」とついているが、プログラミング言語のようなものというよりも、コマンドのようなものをイメージしたほうが合致する。例えば、図 8 のような記述が SQL の問い合わせ文になる。

```

SELECT 書名, 発行年 FROM 図書データ
WHERE ジャンル=ミステリー AND 発行年 BETWEEN 1990 AND 1999

```

図 8 問い合わせ SQL 文例

その問い合わせによって表 1 のような結果が返ってくる。

表 1 問い合わせ結果例

書名	発行年
謎解きはランチの後で	1993
真冬の方程式	1998
8 枚のトンカツ	1991

(4) MySQL

MySQL は、1979 年にスウェーデンの TcX DataKonsult 社で Michael “Monty” Widenius 氏が開発した UNIREG というデータベースがもとになっている。1995 年に MySQL バージョン 1.0 が開発され、1996 年に一般に公開された。UNIX、Windows など、多様なプラットフォームにて利用することができる。

MySQL のライセンスは、GPL とコマーシャルライセンスの 2 種類がある。GPL の場合には、MySQL を用いる、開発したソフトウェアについても GPL に従う必要が出てくる可能性がある。ソフトウェアの改変や再配布について制限したい場合には、コマーシャルライセンスを購入する必要がある。システム自体に組み込むか、別個に立てたデータベースとして使うかどうか、サポートを必要とするかによって、ライセンスやエディションを選ぶ必要がある。

2.5 データベースの構造

リレーショナルデータベースを活用するために必要とするデータベースの構造を確認する。

(1) データベース

データベースは、データを格納する各種テーブルを含むための大きな枠組みである。したが

って、データベース自体はデータを格納するテーブルをまとめるフォルダのようなものとして考えて良いかもしれない。

データベースは、複数のテーブルを持つことができる。例えば、「my_database」という 1 件のデータベースを作成したとき、その中に、my_table1, my_table2, my_table3 の、複数のテーブルが含まれている。

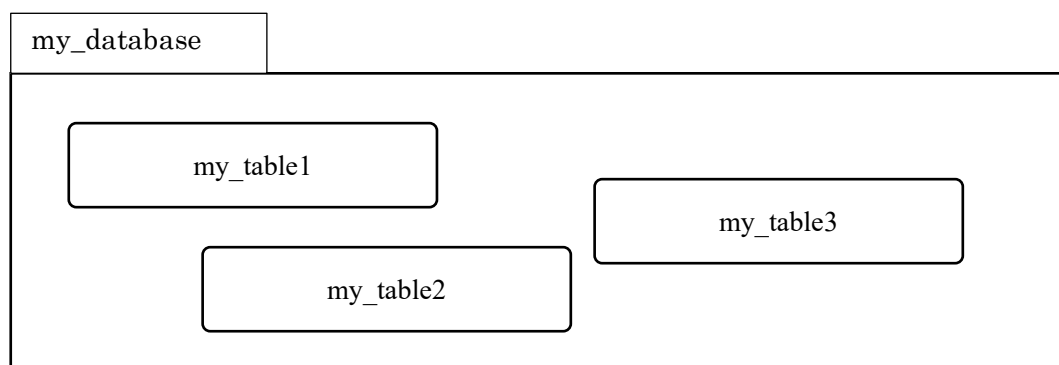


図 9 データベースに含まれるテーブル

例えば、「tshirts_sales_db」というデータベースに「goods」「sales」「customer」というテーブルが含まれている時、図 10 のように、データが格納されていることが確認できる。

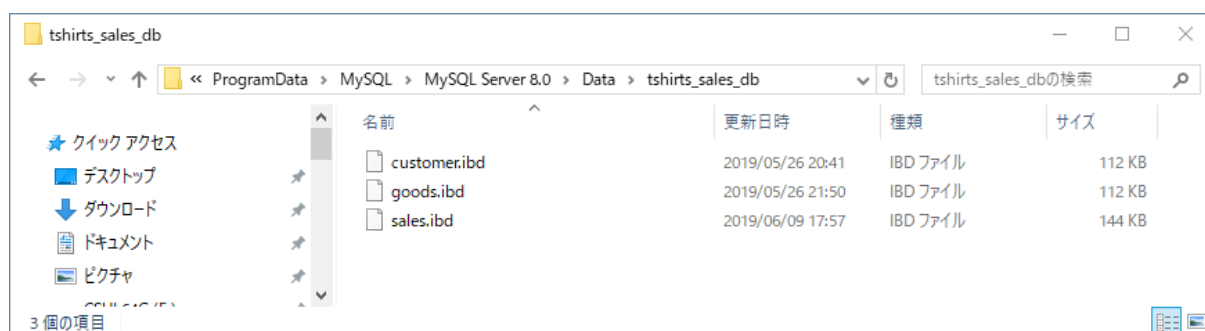


図 10 システム上におけるデータベースとテーブルの様子

作成するテーブルには、ストレージエンジンが設定できる。InnoDB は、信頼性の高いストレージエンジンであり、MySQL では、標準のストレージエンジンである。MySQL 独自の MyISAM は、検索が速いなど、用途や特徴に応じて選ぶことができる。

テーブル型	トランザクション	外部キー	BLOG/TEXT
InnoDB	○	○	○
MyISAM	×	×	○

いずれのデータエンジンが使えるか、SQL から確認できる。

(2) テーブル

テーブルは、データの項目となる、カラム（フィールド）が設定されている。カラムの設定は、テーブルを作成するときに定義する。カラムには、データ型が定義されている。データ型には、数値型、日付型、文字列型などがある。入力されるデータの大きさや範囲に応じて、適切なデータ型を選択する必要がある。

フィールド・カラム				
学籍番号	名前	コース	科目	点数
22421	田中 次郎	情報コース	国語	80
22310	中村 花子	電気コース	物理	95
レコード				

テーブルは、データベースの中に含まれるので、対象とするデータベースを選択している状態で、テーブルを作成する。テーブルを作るときには、テーブルの列のデータ型を指定しなければならない。

また、カラムの制約などを定義するオプションを設定することができる。項目によっては、データを特定するカギ（キー）とするために、空白であることを妨げるようにしたり、自動的に付番するように設定することができる。

次に、数値型、日付型、文字列型に関して、よく使う代表的なデータ型を示す。

(a) 数値型

型	ストレージ (バイト)	最小値	最大値
		(符号付き/符号なし)	(符号付き/符号なし)
TINYINT	1	-128	127
		0	255
SMALLINT	2	-32768	32767
		0	65535
MEDIUMINT	3	-8388608	8388607
		0	16777215
INT	4	-2147483648	2147483647
		0	4294967295
BIGINT	8	-9223372036854775808	9223372036854775807
		0	18446744073709551615

(b) 日付型

型	内容	型/範囲
DATE	日付部分を含むが時間部分は含まない	'YYYY-MM-DD' '1000-01-01' ~ '9999-12-31'
DATETIME	日付と時間の両方の部分を含む	'YYYY-MM-DD HH:MM:SS' '1000-01-01 00:00:00' ~ '9999-12-31 23:59:59'
TIMESTAMP	日付と時間の両方の部分を含む	'1970-01-01 00:00:01' UTC ~ '2038-01-19 03:14:07' UTC

(c) 文字列型

型	内容	範囲
CHAR	固定文字列	0 から 255 までの任意の値
VARCHAR	可変長文字列	0 から 65,535 までの値
BLOB	バイナリ文字列(バイトの文字列)	L + 2 バイト、ここで L < 216
TEXT	非バイナリ文字列(文字の文字列)	L + 2 バイト、ここで L < 216

※ここで、L は指定された文字列値の実際の長さのバイト数を表す。

(d) オプション

オプション	意味
NOT NULL	“NULL”の設定を禁止する
AUTO_INCREMENT	新しい行に一意的識別子をつける
DEFAULT	標準の値を設定する
PRIMARY_KEY(index_column_name)	主キー制約を設定する.

例として、いくつかのフィールドを含むテーブルを作る SQL 文を以下に示す.

```
CREATE TABLE members (id VARCHAR(10) NOT NULL, name VARCHAR(10), gakka INT);
```

設定されたカラムに合致した、データのことを「レコード」と呼ぶ. レコードは、カラムに応じて、1行で表示される1件1件を指す.

テーブルは、1テーブルにたくさんのカラムがある形を思い浮かべるかもしれないが、データベースでは、重複を避けたり、統一を取ったり、集計しやすくしたりするために、複数のテーブルで構成するように、テーブルを関係づけてデータを格納する. 例えば、図 11 と図 12 のように、単独で構成するテーブルから、関係づけられた複数のテーブルで構成するように設計する. これによって、データの統一がはかられ、データの追加や変更などに強くなる.

学籍番号 (id)	名前 (name)	コース
22421	田中 次郎	情報コース
23315	中村 花子	電子コース

図 11 単独のテーブルでの構成

学籍番号 (id)	名前 (name)	コース番号
22421	田中 次郎	2
23315	中村 花子	1

コース番号	コース名称
1	電気コース
2	情報コース

図 12 関係を持つ複数のテーブルでの構成

3. データベースの構築

3.1 データの関係を考える

T シャツの販売管理をするデータベースを作ること考える．整理すべきデータ項目と，関連を考える．



図 13 データベースの関係を考えるための図

3.2 データのコード化

データのことを考えて整理したデータと，それを構成する項目について，データベースのテーブルで関係づけることを前提にして，リストアップして，関係づける．

(a) 顧客台帳

顧客コード
名前
郵便番号
住所
電話番号
メールアドレス

(b) 商品台帳

商品コード
商品名
サイズ
色
単価
在庫数

(c) 売上台帳

売上年月日
<商品コード>
<顧客コード>
数量

3.3 テーブルと関連の図式化

データベースの設計では、テーブルの分け方、テーブルに含むカラムの項目立て、カラムの関連付けが課題になる。それを図で表す便利な方法として、ER 図 (Entity-Relationship 図) がある。ER 図は、テーブルの構造を定義し、テーブル間の関係も表すことができる。この定義や関係は、テーブルを作成するときの SQL コードの形で表すことができる。

astah を使って、テーブルを定義し、関係づけて、テーブル生成の SQL コードを出力することもできる。その具体的な方法は別の章にまとめて示す。

(1) 論理モデル

テーブル名やカラム名など、登場する項目を現実のものそのままを用いて表したモデルを「論理モデル」という。図 14 のように、テーブル名や項目が日本語で表されている。このような、分析などの際に与える、現実の名称などのことを「論理名」と呼ぶ。

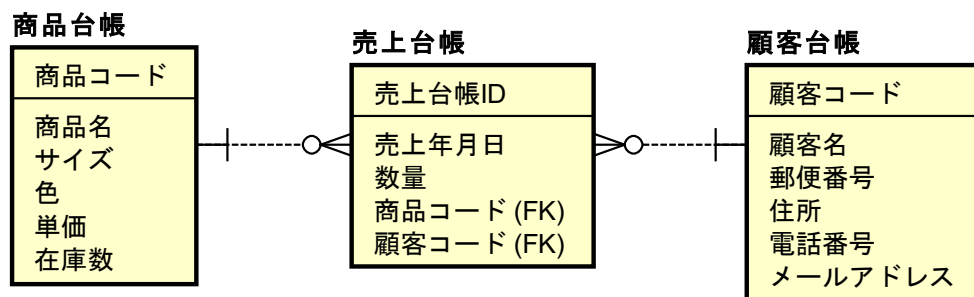


図 14 論理モデル

(2) 物理モデル

実際にデータベースを構成するテーブルを作るときには、この論理名では文字コードなどの制限などから扱いにくいことが多い。プログラムに組み込むときに、日本語の文言が含まれたりすると、文字コードのトラブルに遭遇することも増える。そのため、テーブルのカラム名は英数字で構成される形にする。このように、データベースのテーブルに適用する名前のことを物理名と呼ぶ。物理名で構成されているモデルを「物理モデル」と呼ぶ。

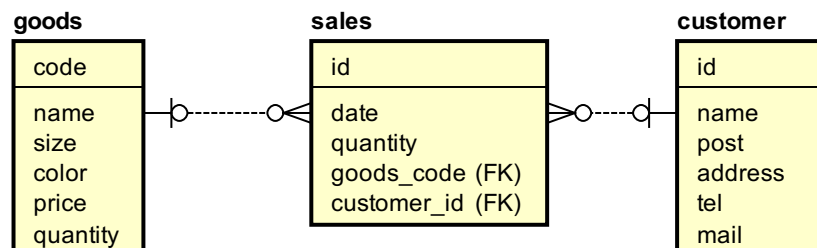
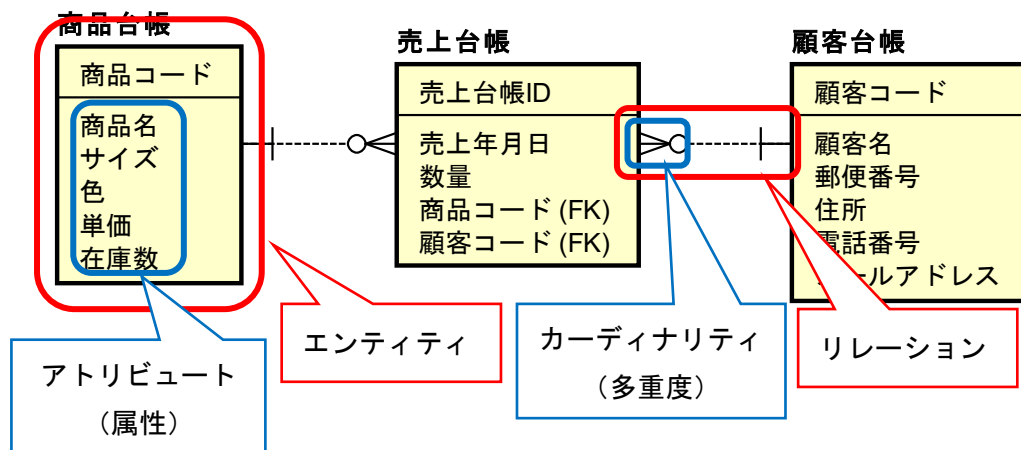


図 15 物理モデル

(3) 構成要素



(a) エンティティ

テーブルを表す，データのまとまりの部品のことである．

(b) アトリビュート

エンティティに含まれる，データ定義のことである．エンティティの属性である．主キーやなども含まれる．主キーは上の領域に位置する．

(c) リレーション

エンティティ同士の関係を表す．依存関係と非依存関係がある．依存関係は，実線で表され，外部の属性が主キーになるときの関係である．非依存関係は，破線で表され，外部のキーが主キーにならない場合に用いる．

(d) カードィナリティ

関係を持つ対象を含む回数によって，記号を用いて表す．

記号	意味
—○	0 または 1
≧○	1 以上
≧○	0 または 1 以上

この授業の範囲では，カードィナリは深く取り扱わない．テーブルを定義する SQL 文が出力できる範囲でとりあつかう．

3.4 テーブル設計

各テーブルについて、カラムの論理名、物理名、データ型、オプションを検討する。

(1) 顧客台帳(customer)

論理名	物理名	データ型	オプション
顧客コード	id	INT	NOT NULL, PRIMARY KEY
名前	name	VARCHAR(256)	
郵便番号	post	CHAR(10)	
住所	address	VARCHAR (256)	
電話番号	tel	CHAR (15)	
メールアドレス	mail	VARCHAR (50)	

(2) 商品台帳(goods)

論理名	物理名	データ型	オプション
商品コード	code	CHAR (32)	NOT NULL, PRIMARY KEY
商品名	name	VARCHAR (40)	
サイズ	size	CHAR (4)	
色	color	CHAR (6)	
単価	price	INT	
在庫数	quantity	INT	

(3) 売上台帳(sales)

論理名	物理名	データ型	オプション
売上台帳 ID	id	CHAR (10)	NOT NULL, PRIMARY KEY
売上年月日	date	DATE	
<商品コード>	goods_code	CHAR (32)	FOREIGN KEY(goods.code)
<顧客コード>	customer_id	INT	FOREIGN KEY(customer.id)
数量	quantity	INT	

3.5 データベース作成と接続

```
CREATE DATABASE tshirts_sales_db;  
USE tshirts_sales_db;
```

3.6 テーブル作成

(a) 顧客台帳(customer)

```
CREATE TABLE customer (  
  id INT NOT NULL PRIMARY KEY,  
  name VARCHAR(256),  
  post CHAR(10),  
  address VARCHAR(256),  
  tel CHAR(15),  
  mail VARCHAR(50)  
);
```

(b) 商品台帳 (goods)

```
CREATE TABLE goods (  
  code CHAR(32) NOT NULL PRIMARY KEY,  
  name VARCHAR(40) NOT NULL,  
  size CHAR(4),  
  color CHAR(6),  
  price INT,  
  quantity INT  
);
```

(c) 売上台帳(sales)

```
CREATE TABLE sales (  
  id CHAR(10) NOT NULL PRIMARY KEY,  
  date DATE,  
  quantity INT,  
  goods_code CHAR(32),  
  customer_id INT,  
  
  FOREIGN KEY (goods_code) REFERENCES goods (code),  
  FOREIGN KEY (customer_id) REFERENCES customer (id)  
);
```

(2) データ入力

(a) 顧客台帳

顧客コード	名前	郵便番号	住所	電話番号	メールアドレス
101	青森商事	030-8570	青森県青森市長島一丁目 1-1	017-722-1111	webmaster@pref.aomori.lg.jp
102	秋田産業	010-8570	秋田県秋田市米区 102-1	018-860-1111	info@mail2.pref.akita.jp
103	岩手観光	020-8570	岩手県盛岡市内丸 10-1	019-651-3111	kouhou@pref.iwate.jp
104	山形販売	990-8570	山形県山形市松波二丁目 8-1	023-630-2211	t57W16uF@pref.yamagata.jp
105	福島ストア	960-8065	福島県福島市杉妻町 2-16	024-521-1111	kouho@pref.fukushima.lg.jp

これらを挿入する命令は以下である。

```
INSERT INTO customer(id,name,post,address,tel,mail) VALUES
(101,"青森商事","030-8570","青森県青森市長島一丁目 1-1","017-722-1111","webmaster@pref.aomori.lg.jp"),
(102,"秋田産業","010-8570","秋田県秋田市米区 102-1","018-860-1111","info@mail2.pref.akita.jp"),
(103,"岩手観光","020-8570","岩手県盛岡市内丸 10-1","019-651-3111","kouhou@pref.iwate.jp"),
(104,"山形販売","990-8570","山形県山形市松波二丁目 8-1","023-630-2211","t57W16uF@pref.yamagata.jp"),
(105,"福島ストア","960-8065","福島県福島市杉妻町 2-16","024-521-1111","kouho@pref.fukushima.lg.jp");
```

(b) 商品台帳

商品コード	商品名	サイズ	色	単価	在庫数
A341	クール T シャツ	L	FF00FF	1980	10
A342	クール T シャツ	M	AA5588	3450	14
B1001	速暖シャツ	S	00FF00	890	3
KK58	高校指定シャツ	M	FA7F00	2500	50
12A4561	速乾 T シャツ	LL	FFFFFF	1280	250

これらを挿入する命令は以下である。

```
INSERT INTO goods(code, name, size, color, price, quantity) VALUES
("A341","クール T シャツ","L","FF00FF","1980","10"),
("A342","クール T シャツ","M","AA5588","3450","14"),
("B1001","速暖シャツ","S","00FF00","890","3"),
("KK58","高校指定シャツ","M","FA7F00","2500","50"),
("12A4561","速乾 T シャツ","LL","FFFFFF","1280","250");
```

(c) 売上台帳

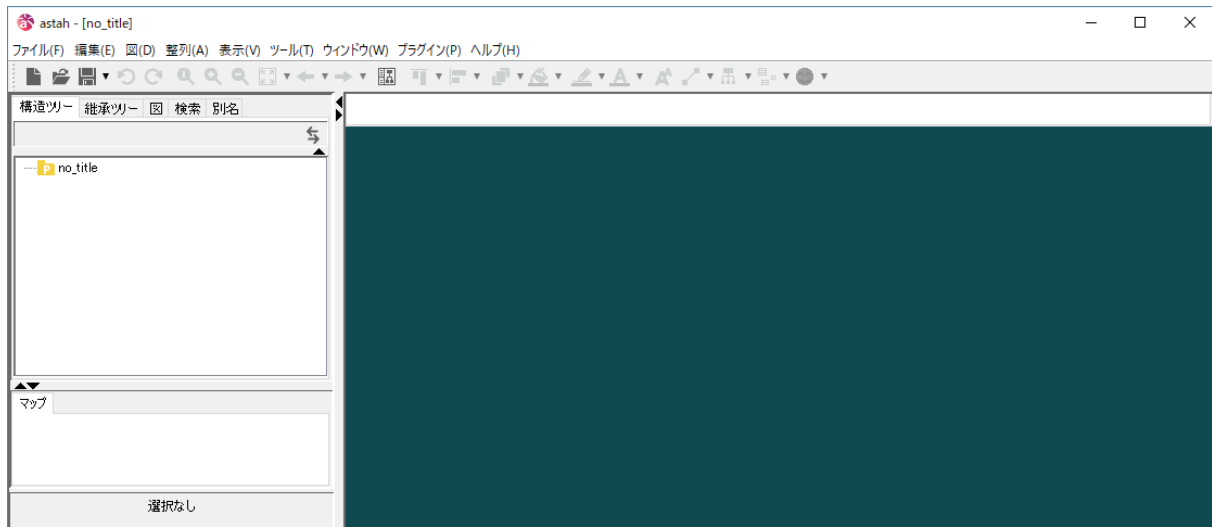
id	売上年月日	＜商品コード＞	＜顧客コード＞	数量
10001	2010/2/13	A341	101	120
10002	2011/4/12	KK58	103	10
10003	2011/5/23	KK58	104	45
10004	2012/10/19	12A4561	102	22
10005	2018/12/24	B1001	103	30

これらを挿入する命令は以下である.

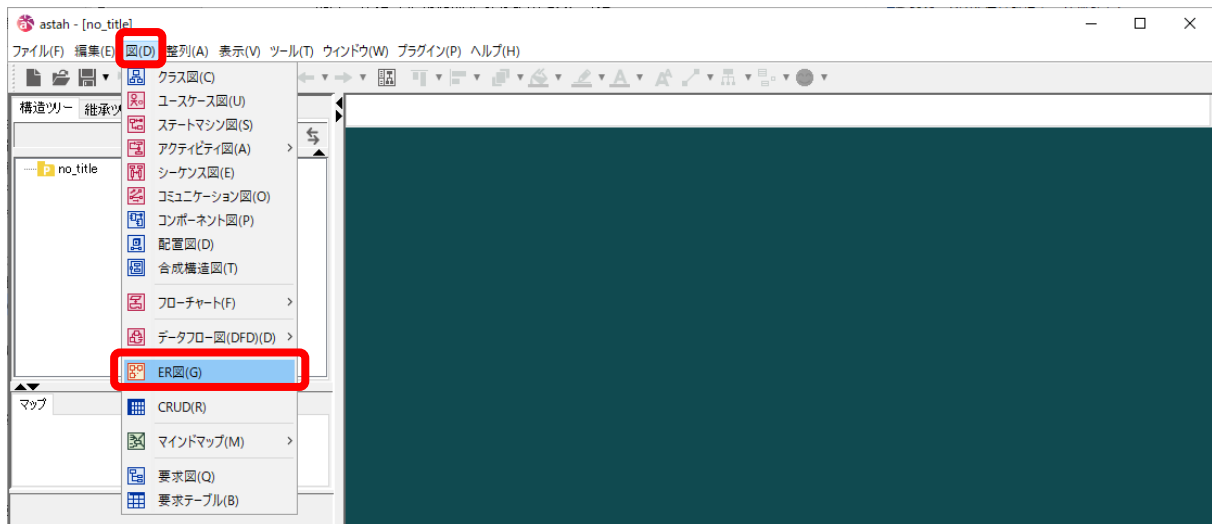
```
INSERT INTO sales(id, date, goods_code,customer_id, quantity) VALUES
("10001","2010/2/13","A341","101","120"),
("10002","2011/4/12","KK58","103","10"),
("10003","2011/5/23","KK58","104","45"),
("10004","2012/10/19","12A4561","102","22"),
("10005","2018/12/24","B1001","103","30");
```


4. ER 図の作成

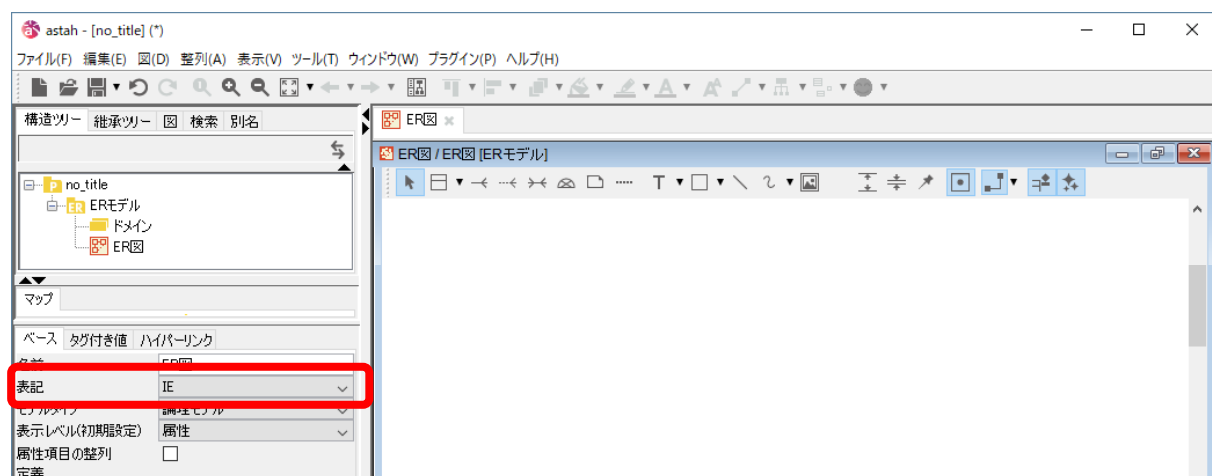
4.1 astah を立ち上げる



4.2 ER 図を新規作成する

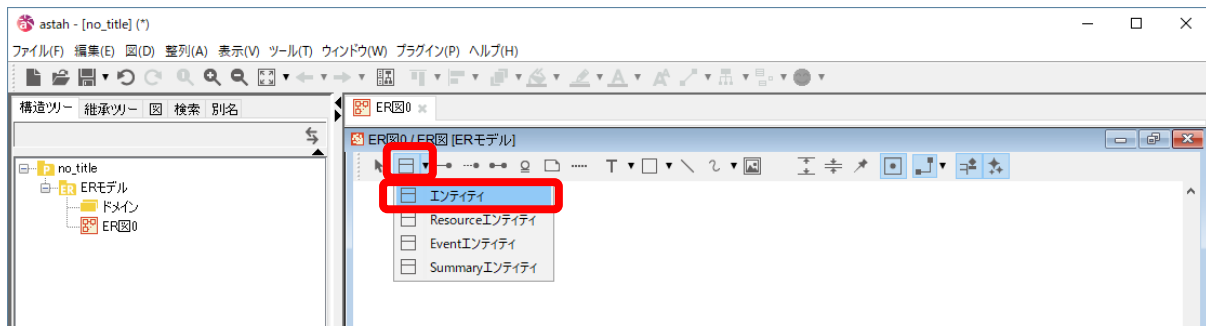


左側の設定項目にて「表記」を「IDEF1X」から「IE」に変更する。



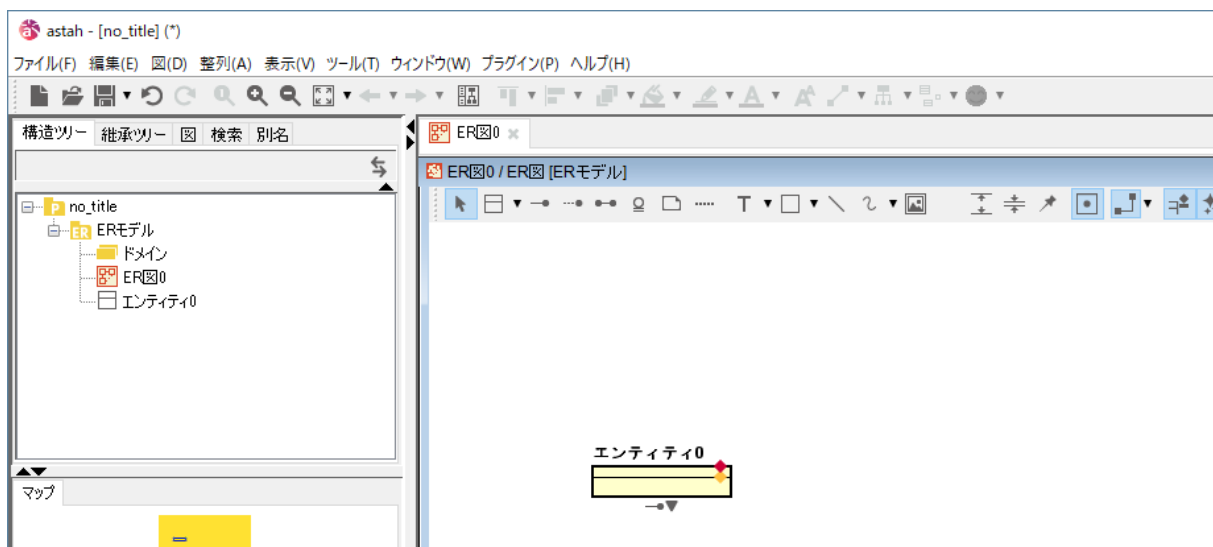
4.3 エンティティ

(1) エンティティを追加する

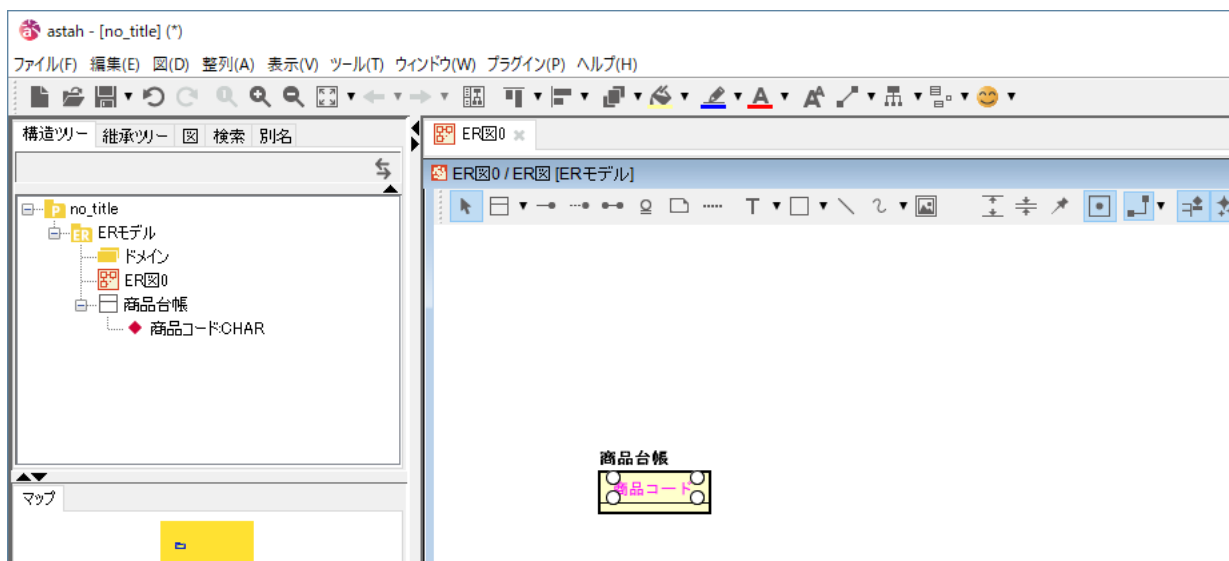


(2) 追加されたエンティティ

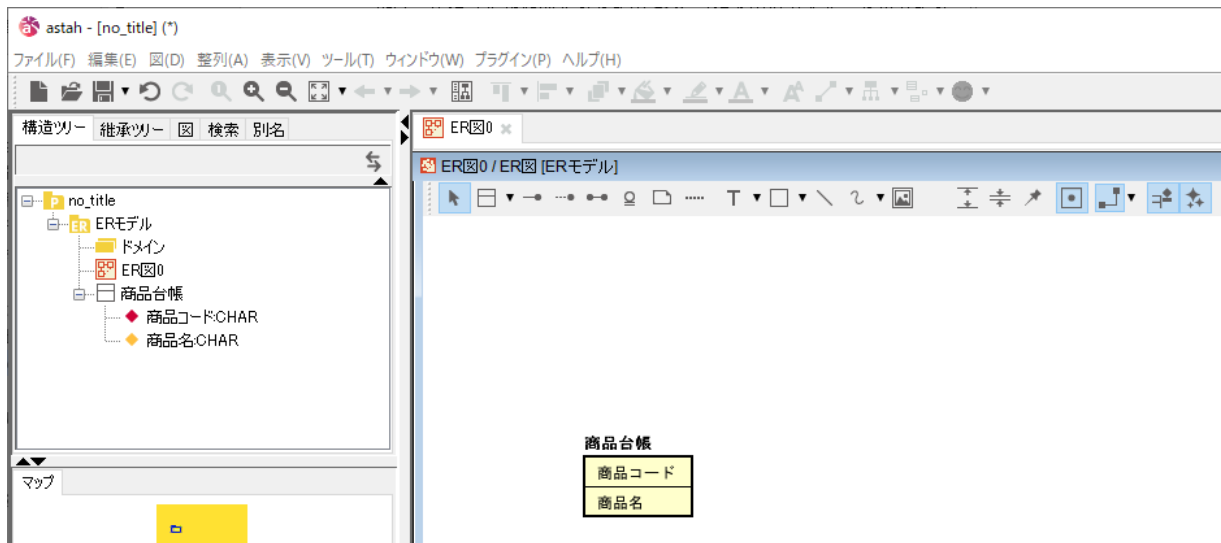
赤いダイヤ◆は主キーを追加するときのアイコンであり、黄色いダイヤ◆は属性を追加するときのアイコンである。



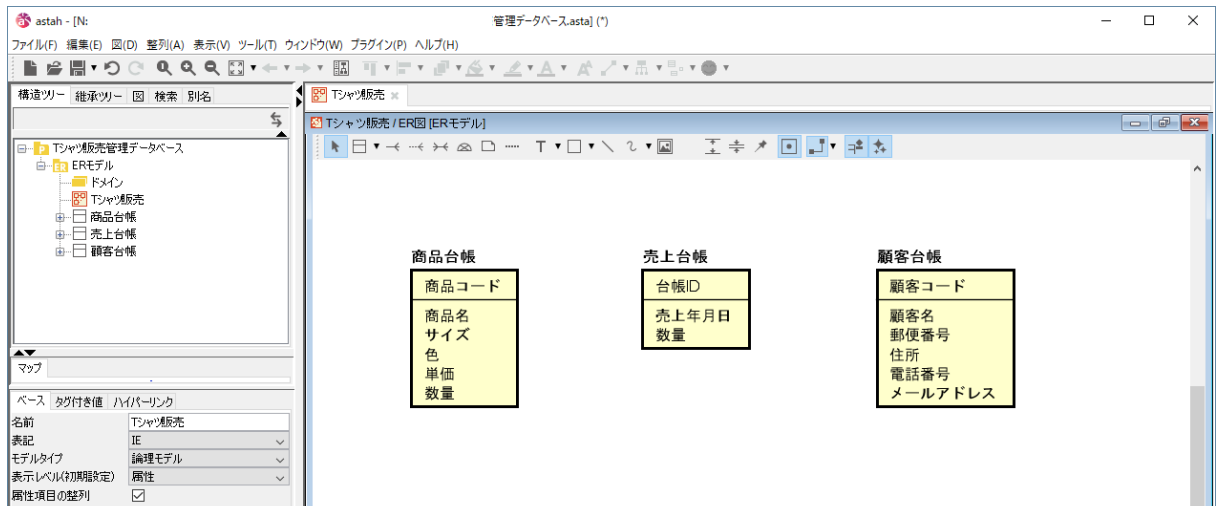
(3) 主キーを追加する



(4) 属性を追加する



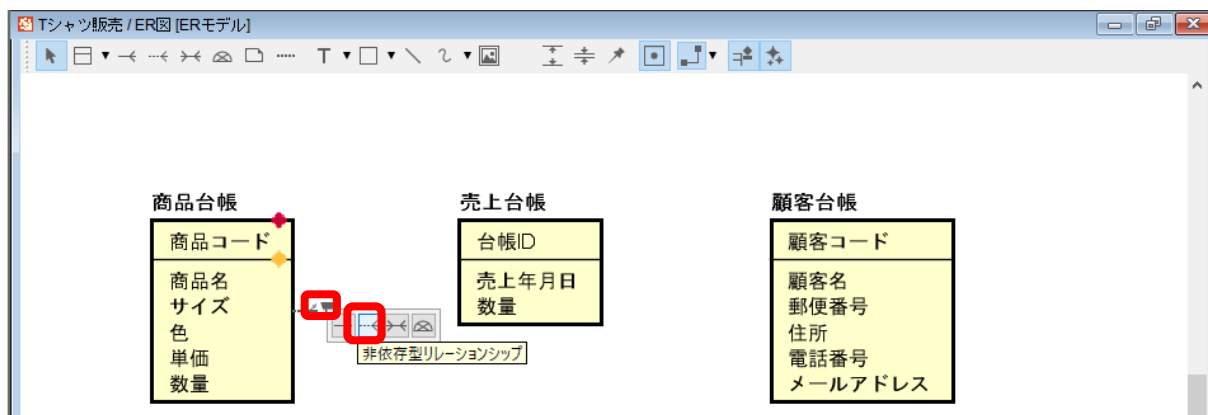
(5) 他のテーブル, 論理名を追加する



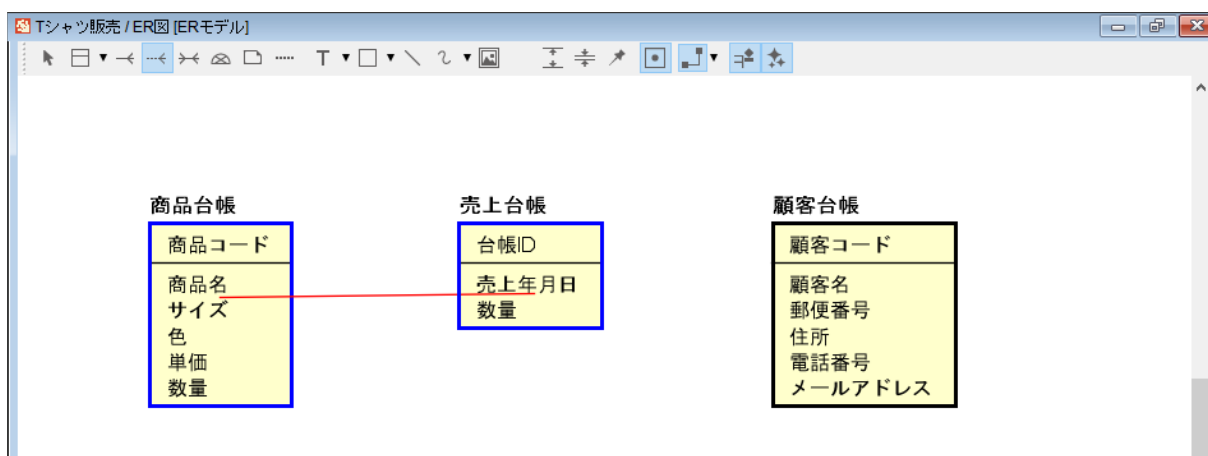
4.4 リレーション

(1) リレーションの接続元

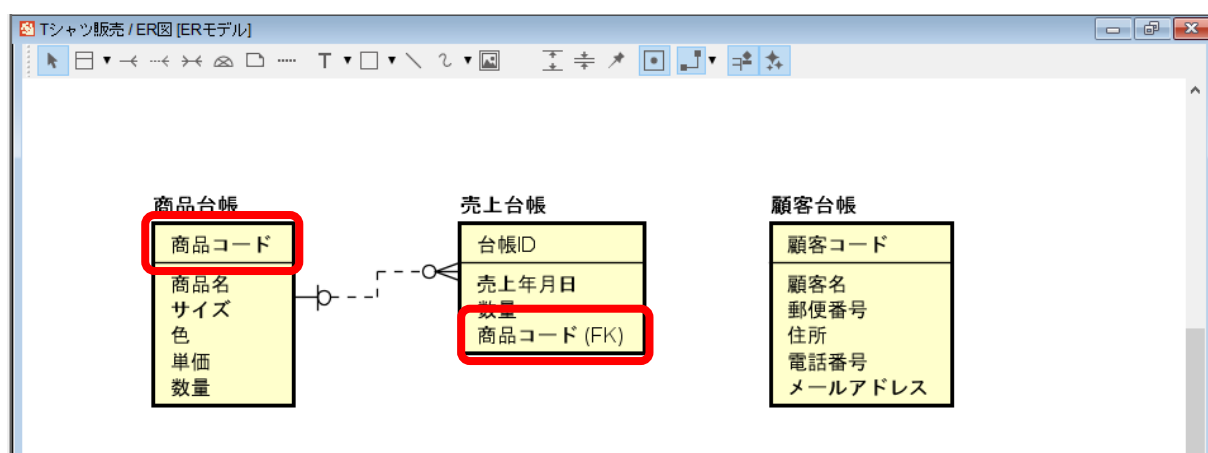
元になるテーブルの接続口から関係を選ぶ。今回のテーブルの関係では、「非依存型リレーションシップ」を選ぶ。



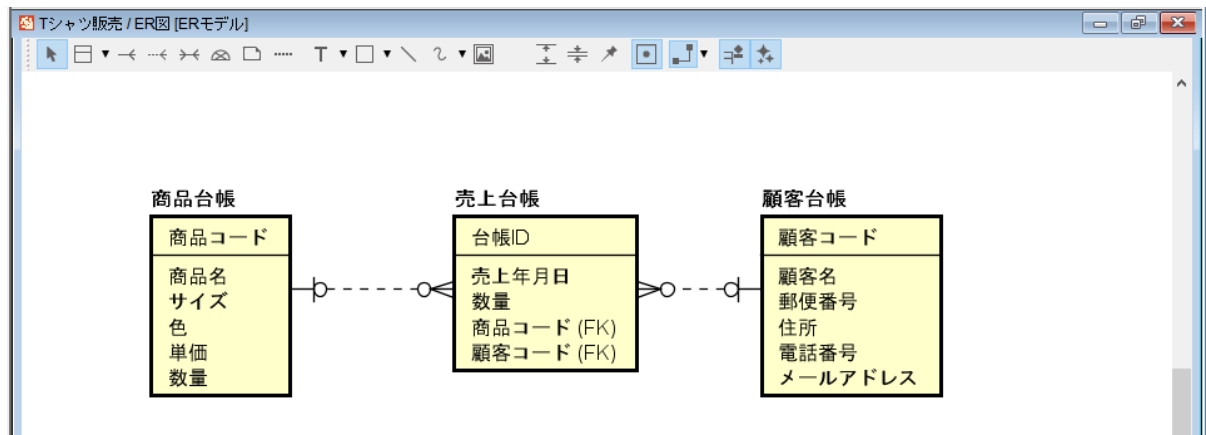
(2) 接続先



接続すると、以下ようになる。方向があるので気をつける。接続元のテーブルの主キーが、接続先の属性に現れていることがわかる。「依存型リレーションシップ」にすると、実線で表され、接続先の主キーとしてエントリされる。



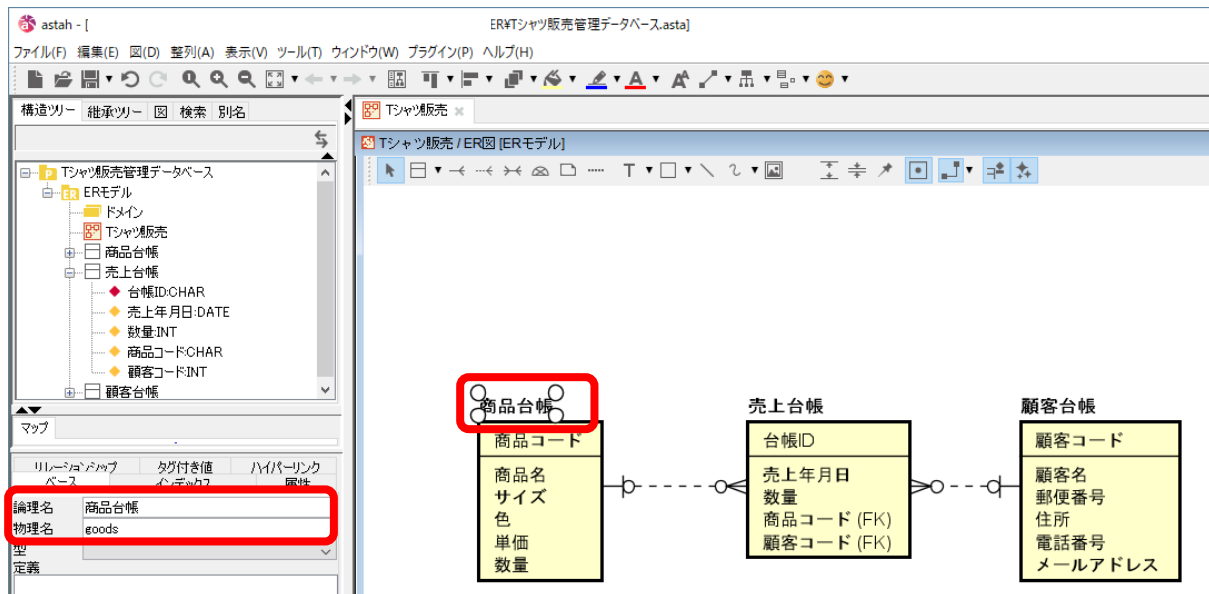
他の接続も関係づける。



4.5 物理名

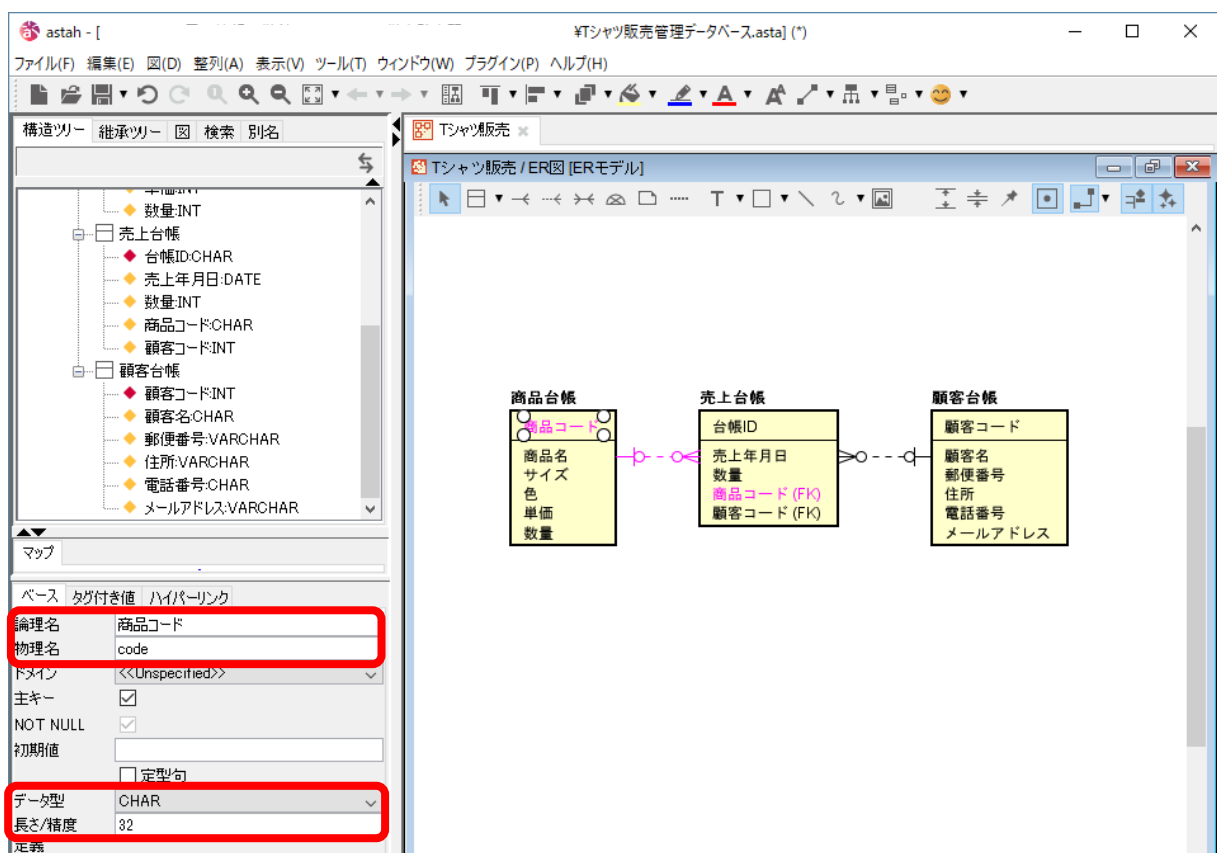
(1) テーブル名

テーブル名をクリックして、左側のプロパティの「物理名」にテーブルの物理名を入力する。



(2) 主キー名

主キーをクリックして、左側の設定にて、物理名やデータ型などを定義する。



(3) 属性名

属性をクリックして、物理名やデータ型などを定義する。

商品台帳

商品コード
色
単価
数量

売上台帳

台帳ID
売上年月日
数量
商品コード (FK)
顧客コード (FK)

顧客台帳

顧客コード
顧客名
郵便番号
住所
電話番号
メールアドレス

ベース

論理名	商品名
物理名	name
データ型	VARCHAR
長さ/精度	40

(4) 物理モデルを表示する

プロパティの「ベース」タブにある「モデルタイプ」を選択できるので、「物理モデル」に切り替えると、物理モデルが表示される。ここで括弧付きで「(論理名)」のように表示されると、物理名が設定されていないので、もれなく設定する。

goods

code
name
size
color
price
quantity

sales

id
date
quantity
goods_code (FK)
customer_id (FK)

customer

id
name
post
address
tel
mail

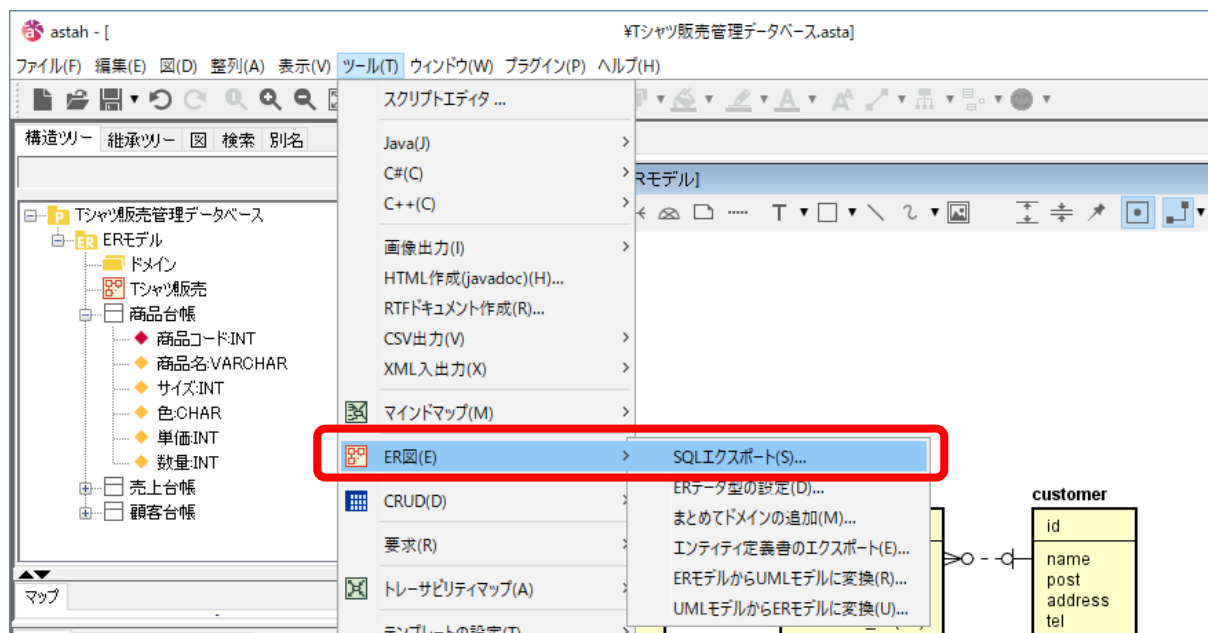
ベース

名前	Tシャツ販売
モデルタイプ	物理モデル

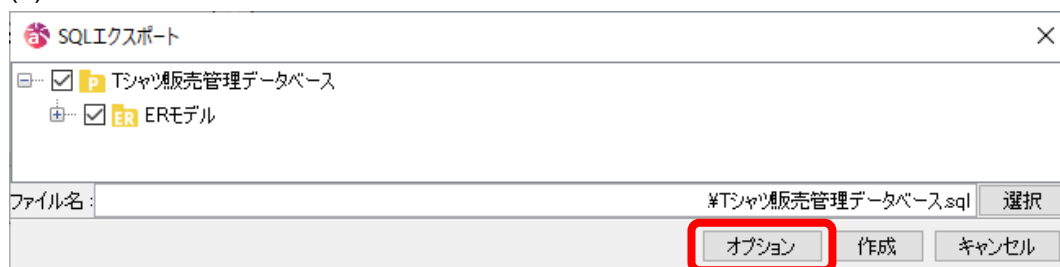
4.6 SQL 出力

(1) SQL エクスポート

「ツール／ER 図／SQL エクスポート」を選ぶ。



(2) 「オプション」を選択する



(3) 「CREATE TABLE 文のみ使用する」にチェック

チェックを入れると、外部キーの設定が CREATE TABLE 文に含めた形で出力される。

SQLエクスポートオプションの設定

出力するモデルタイプ: 物理モデル

☐ カラム名を引用符文字列で囲む

☒ CREATE TABLE文のみを使用する

☐ DROP TABLE文を出力する

☒ 外部キーの出力

DROP TABLE: DROP TABLE文にCASCADE CONSTRAINTSを出力する

一意インデックス: CREATE UNIQUE INDEX文を出力する

☒ 非一意インデックス

☐ セパレータ文字列: :

コメント: エンティティ: なし, 属性: なし

ファイル出力時の文字コード: UTF-8

了解 キャンセル

(4) 出力する

SQLエクスポート

Database: Tシャツ販売管理データベース

Model: ERモデル

ファイル名: #Tシャツ販売管理データベース.sql

選択

オプション 作成 キャンセル

Information

エクスポートが正常終了しました。

OK

5. データベースの設計

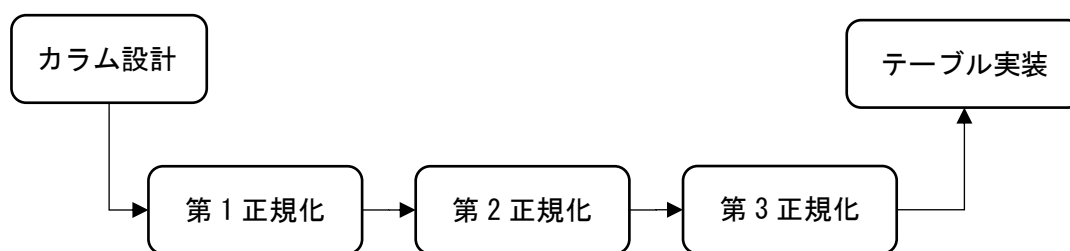


図 16 テーブル設計の手順

(1) フィールドの洗い出し

正規化されていない，非正規形のテーブルを表 2 に示す．

表 2 非正規形テーブル

学籍番号	学年	名前	系	科目	担当者	点数
22421	4	田中 次郎	情報エレクトロニクス系	データベース	中村 剛	90
				電子回路	和田 知子	100
22310	4	中村 花子	機械ロボティクス系	制御工学	江田 純也	87
				材料力学	藤枝 明彦	68
22315	4	成田 翼	機械ロボティクス系	材料力学	藤枝 明彦	89
21504	3	木村 剛	都市デザイン系	都市計画	内川 早苗	89

(2) 第 1 正規化

レコードをまたいで定義されている値が無いように分け，1 レコードごとに分離する．第 1 正規化したテーブルを表 3 に示す．

表 3 第 1 正規形テーブル

学籍番号	学年	名前	系	科目	担当者	点数
22421	4	田中 次郎	情報エレクトロニクス系	データベース	中村 剛	90
22421	4	田中 次郎	情報エレクトロニクス系	電子回路	和田 知子	100
22310	4	中村 花子	機械ロボティクス系	制御工学	江田 純也	87
22310	4	中村 花子	機械ロボティクス系	材料力学	藤枝 明彦	68
22315	4	成田 翼	機械ロボティクス系	材料力学	藤枝 明彦	89
21504	3	木村 剛	都市デザイン系	都市計画	内川 早苗	89

(3) 第 2 正規化

主となるカラムとそれに従ってきまる従となるカラムを分離する．例えば，学籍番号を主として，学年や氏名，所属学科が決まる．また，科目を主として，その教科担当が決まる．さらに，点数は，学籍番号に応じた科目の点数として決まる．したがって，以下のように，3 種類のテーブルに分けられる．

主キー

学籍番号	学年	名前	系	科目	担当者	点数
22421	4	田中 次郎	情報エレクトロニクス系	データベース	中村 剛	90
22421	4	田中 次郎	情報エレクトロニクス系	電子回路	和田 知子	100
22310	4	中村 花子	機械ロボティクス系	制御工学	江田 純也	87
22310	4	中村 花子	機械ロボティクス系	材料力学	藤枝 明彦	68
22315	4	成田 翼	機械ロボティクス系	材料力学	藤枝 明彦	89
21504	3	木村 剛	都市デザイン系	都市計画	内川 早苗	89

主キー

表 4 学生に関するデータ

学籍番号	学年	名前	系
22421	4	田中 次郎	情報エレクトロニクス系
22421	4	田中 次郎	情報エレクトロニクス系
22310	4	中村 花子	機械ロボティクス系
22310	4	中村 花子	機械ロボティクス系
22315	4	成田 翼	機械ロボティクス系
21504	3	木村 剛	都市デザイン系

表 5 科目に関するデータ

科目	担当者
データベース	中村 剛
電子回路	和田 知子
制御工学	江田 純也
材料力学	藤枝 明彦
材料力学	藤枝 明彦
都市計画	内川 早苗

表 6 点数に関するデータ

学籍番号	科目	点数
22421	データベース	90
22421	電子回路	100
22310	制御工学	87
22310	材料力学	68
22315	材料力学	89
21504	都市計画	89

抜き出したとき、同じ項目になるところがある。それは冗長であるので、表 7 から、表 8 のように削除する。同様に、表 9 から、表 10 のように重複を削除する。

表 7 学生に関するデータ 重複削除前

学籍番号	学年	名前	系
22421	4	田中 次郎	情報エレクトロニクス系
22421	4	田中 次郎	情報エレクトロニクス系
22310	4	中村 花子	機械ロボティクス系
22310	4	中村 花子	機械ロボティクス系
22315	4	成田 翼	機械ロボティクス系
21504	3	木村 剛	都市デザイン系

表 8 学生に関するデータ 重複削除後

学籍番号	学年	名前	系
22421	4	田中 次郎	情報エレクトロニクス系
22310	4	中村 花子	機械ロボティクス系
22315	4	成田 翼	機械ロボティクス系
21504	3	木村 剛	都市デザイン系

表 9 科目に関するデータ 重複削除前

科目	担当者
データベース	中村 剛
電子回路	和田 知子
制御工学	江田 純也
材料力学	藤枝 明彦
材料力学	藤枝 明彦
都市計画	内川 早苗

表 10 科目に関するデータ 重複削除後

科目	担当者
データベース	中村 剛
電子回路	和田 知子
制御工学	江田 純也
材料力学	藤枝 明彦
都市計画	内川 早苗

第 2 正規化は第 3 正規化のための途中経過であるので、それほど厳密に決まっているわけではない。

(4) 第 3 正規化

第 2 正規化で分離できていない、従属している部分を抜き出す。部分的に、いくつか選択肢があつて、その中から選ぶような項目があれば、その部分を別のテーブルに抜き出す。

例えば、学生のテーブルにおける「学科」については、学生によって異なるというよりも、一定の選択肢から選ぶ内容である。学科を別の表にまとめる。抜き出した後の部分は、後で関係づけるために、ここでは、このままにしておく。

学籍番号	学年	名前	系	系
22421	4	田中 次郎	情報エレクトロニクス系	情報エレクトロニクス系
22310	4	中村 花子	機械ロボティクス系	機械ロボティクス系
22315	4	成田 翼	機械ロボティクス系	機械ロボティクス系
21504	3	木村 剛	都市デザイン系	都市デザイン系

抜き出したことで、冗長なデータが生じるので、それも削除しておこう。

学籍番号	学年	名前	系	系
22421	4	田中 次郎	情報エレクトロニクス系	情報エレクトロニクス系
22310	4	中村 花子	機械ロボティクス系	機械ロボティクス系
22315	4	成田 翼	機械ロボティクス系	都市デザイン系
21504	3	木村 剛	都市デザイン系	

(5) テーブル実装

(a) 主キーを与える

主キーとなるカラムがある場合はそのカラムを主キーとすればよいが、そうなれないカラムしかない場合には、テーブルのレコードにユニークなキーとなる id などを設ける。

表 11 学生に関するテーブル

学籍番号	学年	名前	系
22421	4	田中 次郎	情報エレクトロニクス系
22310	4	中村 花子	機械ロボティクス系
22315	4	成田 翼	機械ロボティクス系
21504	3	木村 剛	都市デザイン系

表 12 系に関するテーブル

系 ID	系
1	情報エレクトロニクス系
2	機械ロボティクス系
3	都市デザイン系

表 13 科目に関するテーブル

科目 ID	科目	担当者
10001	データベース	中村 剛
10002	電子回路	和田 知子
10003	制御工学	江田 純也
10004	材料力学	藤枝 明彦
10005	都市計画	内川 早苗

表 14 成績に関するテーブル

学籍番号	科目	点数
22421	データベース	90
22421	電子回路	100
22310	制御工学	87
22310	材料力学	68
22315	材料力学	89
21504	都市計画	89

(b) 外部キーを参照させる

表 15 学生表の外部キー参照

学籍番号	学年	名前	系
22421	4	田中 次郎	情報エレクトロニクス系
22310	4	中村 花子	機械ロボティクス系
22315	4	成田 翼	機械ロボティクス系
21504	3	木村 剛	都市デザイン系



学籍番号	学年	名前	系 ID
22421	4	田中 次郎	1
22310	4	中村 花子	2
22315	4	成田 翼	2
21504	3	木村 剛	3

表 16 参照される系表

系 ID	系
1	情報エレクトロニクス系
2	機械ロボティクス系
3	都市デザイン系

表 17 点数表の外部キー参照

学籍番号	科目	点数
22421	データベース	90
22421	電子回路	100
22310	制御工学	87
22310	材料力学	68
22315	材料力学	89
21504	都市計画	89



学籍番号	科目 ID	点数
22421	10001	90
22421	10002	100
22310	10003	87
22310	10004	68
22315	10004	89
21504	10005	89

表 18 参照される科目表

科目 ID	科目	担当者
10001	データベース	中村 剛
10002	電子回路	和田 知子
10003	制御工学	江田 純也
10004	材料力学	藤枝 明彦
10005	都市計画	内川 早苗

(c) カラムを定義する

表 19 学生表(student)のカラム設定

論理名	物理名	データ型	オプション
学籍番号	id	INT	NOT NULL PRIMARY KEY
学年	grade	INT	
名前	name	CHAR(32)	
系	course_id	INT	FOREIGN KEY(course.id)

表 20 系表(course)のカラム設定

論理名	物理名	データ型	オプション
系 ID	id	INT	NOT NULL PRIMARY KEY
系	course	CHAR (64)	

表 21 点数表(score)のカラム設定

論理名	物理名	データ型	オプション
学籍番号	id	INT	NOT NULL PRIMARY KEY
科目	subject_id	INT	NOT NULL PRIMARY KEY FOREIGN KEY(subject.id)
点数	score	INT	

表 22 科目表(subject)のカラム設定

論理名	物理名	データ型	オプション
科目 ID	id	INT	NOT NULL PRIMARY KEY
科目	subject	VARCHAR(128)	
担当者	teacher	CHAR (32)	

6. SQL

6.1 データベース作成

「database_name」という名称のデータベースを作るときには、以下の命令を実行する。

```
> CREATE DATABASE database_name;
```

「company」というデータベースを作成するときには、以下のようになる。

```
mysql> CREATE DATABASE company;
```

どんなデータベースがあるのか確認するには「show」コマンドを用いる。以下のように指定することで、今有るデータベースを一覧で確認できる。

```
> SHOW DATABASES;
```

初期状態で実行すると以下のようになる、

```
mysql> SHOW DATABASES;
+-----+
| Database |
+-----+
| information_schema |
| mysql       |
| performance_schema |
| sys         |
+-----+
```

データベースを作成しただけだと、ただ作っただけで、そのデータベースが操作対象となっていない。そこで、操作する対象とするには、「use」コマンドを使って、対象のデータベースを指定する。

```
> USE database_name;
```

「company」データベースを使うときには、以下のようになる。

```
mysql> USE company;
```

6.2 テーブル作成

```
> CREATE TABLE table_name (col_name1 date_type1, col_name2 date_type2,  
    ...) table_option
```

「member」テーブルを作成するには以下ようになる。

```
> CREATE TABLE member (  
  id int NOT NULL,  
  name VARCHAR (64),  
  place VARCHAR (64),  
  age INT  
);
```

ここで対象としているデータベースに含まれるテーブルの一覧を表示する命令は以下である。

```
> SHOW TABLES;
```

以下が実行例である。

```
> SHOW TABLES;
```

```
+-----+  
| Tables_in_company |  
+-----+  
| member            |  
+-----+
```

テーブルの定義を確認する命令は以下である。

```
> DESC table_name;
```

以下に例を示す。

```
mysql> DESC member;
```

```
+-----+-----+-----+-----+-----+-----+  
| Field | Type      | Null | Key | Default | Extra |  
+-----+-----+-----+-----+-----+-----+  
| id    | int(11)   | NO   |     | NULL    |       |  
| name  | varchar(64) | YES  |     | NULL    |       |  
| place | varchar(64) | YES  |     | NULL    |       |  
| age   | int(11)   | YES  |     | NULL    |       |  
+-----+-----+-----+-----+-----+-----+
```

6.3 挿入

データをテーブルに挿入する。挿入するときのデータの指定の仕方は複数ある。

```
INSERT INTO table_name (col1, col2, ...) VALUES(val1, val2, ...);
```

1 件挿入するとき。

```
INSERT INTO member (id, name, place, age) VALUES(101, '木村太郎', '北海道', 28);
```

複数件挿入するとき。

```
INSERT INTO member (id, name, place, age) VALUES  
(102, '田中花子', '宮城', 38),  
(103, '中村裕子', '宮城', 41);
```

カラムの順番を、登録されている順番とするとき。

```
INSERT INTO member VALUES (104, '中村康太', '青森', 41);
```

これらを実行した結果は以下となる。

```
mysql> SELECT * FROM member;
```

id	name	place	age
101	木村太郎	北海道	28
102	田中花子	宮城	38
103	中村裕子	宮城	41
104	中村康太	青森	41

6.4 選択

データを抽出するときには、SELECT 文を使う。

```
SELECT col1, col2,... FROM table_name WHERE condition
```

表示する項目や順番を指定せずに、「すべて」とするときには、カラムの指定欄に「*」を指定する。

(1) そのまま表示

```
mysql> SELECT * FROM member;
```

id	name	place	age
101	木村太郎	北海道	28
102	田中花子	宮城	38
103	中村裕子	宮城	41
104	中村康太	青森	41

表示する項目を選ぶときや、表示する順番を指定するときは、カラム名を具体的に指定する。

```
mysql> SELECT id,name FROM member;
```

id	name
101	木村太郎
102	田中花子
103	中村裕子
104	中村康太

単独のテーブルの時には問題にならないが、複数のテーブルを扱うときに、カラム名が重複するときに、テーブル名を明示してカラムを指定するときには、以下のようにする。

```
mysql> SELECT id,member.name,member.age FROM member;
```

id	name	age
101	木村太郎	28
102	田中花子	38
103	中村裕子	41
104	中村康太	41

表示するカラム名を指定したいときには、「AS」を使う。

```
mysql> SELECT id,member.name,member.age AS toshi FROM member;
```

id	name	toshi
101	木村太郎	28
102	田中花子	38
103	中村裕子	41
104	中村康太	41

順番を指定したいときには、「ORDER BY」にて指定する。

```
mysql> SELECT * FROM member ORDER BY age;
```

id	name	place	age
101	木村太郎	北海道	28
102	田中花子	宮城	38
103	中村裕子	宮城	41
104	中村康太	青森	41

昇順, 降順を指定しないときには, 昇順であるので, 暗示的に「ASC」を指定していることになる. これを明示的に指定すると, 以下になる.

```
mysql> SELECT * FROM member ORDER BY age ASC;
```

id	name	place	age
101	木村太郎	北海道	28
102	田中花子	宮城	38
103	中村裕子	宮城	41
104	中村康太	青森	41

降順にするには, 「DESC」を指定する.

```
mysql> SELECT * FROM member ORDER BY age DESC;
```

id	name	place	age
103	中村裕子	宮城	41
104	中村康太	青森	41
102	田中花子	宮城	38
101	木村太郎	北海道	28

単独のテーブルを扱っている場合には生じないが, テーブルを結合するときには, 重複する結果が表示されることがある.

```
mysql> SELECT id,name,place,age FROM member;
```

id	name	place	age
104	中村康太	青森	41
102	田中花子	宮城	38
102	田中花子	宮城	38

こんなときには, 「DISTINCT」を指定することで, 重複を除いた結果が表示される.

```
mysql> SELECT DISTINCT id,name,place,age FROM member;
```

id	name	place	age
104	中村康太	青森	41
102	田中花子	宮城	38

(2) 条件

条件を指定するときには、演算子や、ワイルドカードを使うことができる。

表 23 演算子

演算子	例	説明
=	age = 20	age が 20 である
!=	age != 20	age が 20 でない
<	age < 20	age が 20 未満
<=	age <= 20	age が 20 以下
>	age > 20	age が 20 よりも大きい
>=	age >= 20	age が 20 以上

表 24 演算子 2

演算子	例	説明
AND	age = 41 AND place='宮城'	age が 41 かつ place が宮城
OR	age = 41 OR place='宮城'	age が 41 または place が宮城
BETWEEN AND	age BETWEEN 20 AND 30	age が 20 から 30 の間
LIKE	age LIKE 41	age が 41

表 25 ワイルドカード

記号	意味	例	説明
%	0 文字以上の文字列	name LIKE '藤田%'	name が藤田から始まるすべて
_	任意の 1 文字	age LIKE '4_'	age が 4 から始める 2 桁の文字

```
mysql> SELECT id,name,place,age FROM member WHERE age=41;
```

```
+-----+-----+-----+-----+
| id  | name      | place | age  |
+-----+-----+-----+-----+
| 103 | 中村裕子  | 宮城  | 41   |
| 104 | 中村康太  | 青森  | 41   |
+-----+-----+-----+-----+
```

```
mysql> SELECT id,name,place,age FROM member WHERE age=41 AND place='宮城';
```

```
+-----+-----+-----+-----+
| id  | name      | place | age  |
+-----+-----+-----+-----+
| 103 | 中村裕子  | 宮城  | 41   |
+-----+-----+-----+-----+
```

```
mysql> SELECT * FROM member WHERE age>30;
```

id	name	place	age
102	田中花子	宮城	38
103	中村裕子	宮城	41
104	中村康太	青森	41

```
mysql> SELECT * FROM member where name LIKE "中村%";
```

id	name	place	age
103	中村裕子	宮城	41
104	中村康太	青森	41

```
mysql> SELECT * FROM member WHERE age LIKE '_8' ;
```

id	name	place	age
101	木村太郎	北海道	28
102	田中花子	宮城	38

(3) 機能

データを簡単に集計したり，選択したりする機能が備わっている．選択された対象の値の平均値や合計などを計算して表示することができる．

```
mysql> SELECT MAX(age) FROM member;
```

```
+-----+  
| MAX(age) |  
+-----+  
|      41 |  
+-----+
```

```
mysql> SELECT AVG(age) FROM member;
```

```
+-----+  
| AVG(age) |  
+-----+  
|  37.0000 |  
+-----+
```

```
mysql> SELECT SUM(age) FROM member;
```

```
+-----+  
| SUM(age) |  
+-----+  
|      148 |  
+-----+
```

```
mysql> SELECT COUNT(*) FROM member;
```

```
+-----+  
| COUNT(*) |  
+-----+  
|         4 |  
+-----+
```


6.5 削除

(1) データベース削除

```
DROP DATABASE database_name;
```

以下に例を示す.

```
mysql> DROP DATABASE company;
```

これで, 「company」データベースが削除される.

(2) テーブル削除

```
DROP TABLE table_name;
```

以下に例を示す.

```
mysql> DROP TABLE member;
```

これで, 「member」テーブルが削除される.

(3) データ削除

```
DELETE FROM table_name WHERE condition;
```

```
mysql> DELETE FROM member WHERE id=104;
```

```
mysql> SELECT * FROM member;
```

id	name	place	age
101	木村太郎	北海道	28
102	田中花子	宮城	38
103	中村裕子	宮城	41

以下の命令で, 指定したテーブルのすべてのデータが削除できる.

```
DELETE FROM table_name;
```

6.6 更新

既に登録されているレコードを更新するときに使う.

```
UPDATE table_name SET col=val WHERE condition;
```

```
mysql> UPDATE member SET place='岩手' WHERE id='101';
```

```
mysql> SELECT * FROM member;
```

id	name	place	age
101	木村太郎	岩手	28
102	田中花子	宮城	38
103	中村裕子	宮城	41
104	中村康太	青森	41

条件をしっかりと指定しないと, 条件が当てはまるレコードすべてが更新されるので, 気をつける.

6.7 設定変更

データベースの設定を変える SQL 文を見る.

(1) データ型変更

変更前の状態を示す.

```
mysql> DESC member;
```

Field	Type	Null	Key	Default	Extra
id	int(11)	NO		NULL	
name	varchar(64)	YES		NULL	
place	varchar(64)	YES		NULL	
age	int(11)	YES		NULL	

```
ALTER TABLE member MODIFY place char(32);
```

```
mysql> DESC member;
```

Field	Type	Null	Key	Default	Extra
id	int(11)	NO		NULL	
name	varchar(64)	YES		NULL	
place	char(32)	YES		NULL	
age	int(11)	YES		NULL	

(2) カラム追加

追加前.

```
mysql> DESC member;
```

Field	Type	Null	Key	Default	Extra
id	int(11)	NO		NULL	
name	varchar(64)	YES		NULL	
place	char(32)	YES		NULL	
age	int(11)	YES		NULL	

```
ALTER TABLE member add birthday date;
```

追加後.

```
mysql> DESC member;
```

Field	Type	Null	Key	Default	Extra
id	int(11)	NO		NULL	
name	varchar(64)	YES		NULL	
place	char(32)	YES		NULL	
age	int(11)	YES		NULL	
birthday	date	YES		NULL	

(3) カラム名

変更前

Field	Type	Null	Key	Default	Extra
id	int(11)	NO		NULL	
name	varchar(64)	YES		NULL	
place	char(32)	YES		NULL	
age	int(11)	YES		NULL	
birthday	date	YES		NULL	

命令

```
ALTER TABLE member CHANGE name namae varchar(64);
```

変更後

```
mysql> DESC member;
```

Field	Type	Null	Key	Default	Extra
id	int(11)	NO		NULL	
namae	varchar(64)	YES		NULL	
place	char(32)	YES		NULL	
age	int(11)	YES		NULL	
birthday	date	YES		NULL	

(4) カラム削除

削除前

```
mysql> DESC member;
```

Field	Type	Null	Key	Default	Extra
id	int(11)	NO		NULL	
namae	varchar(64)	YES		NULL	
place	char(32)	YES		NULL	
age	int(11)	YES		NULL	
birthday	date	YES		NULL	

```
ALTER TABLE member DROP birthday;
```

削除後

```
mysql> DESC member;
```

Field	Type	Null	Key	Default	Extra
id	int(11)	NO		NULL	
namae	varchar(64)	YES		NULL	
place	char(32)	YES		NULL	
age	int(11)	YES		NULL	

(5) 主キー設定

(a) 単独カラム

```
mysql> DESC member;
```

Field	Type	Null	Key	Default	Extra
id	int(11)	NO		NULL	
name	varchar(64)	YES		NULL	
place	char(32)	YES		NULL	
age	int(11)	YES		NULL	

```
ALTER TABLE member ADD PRIMARY KEY (id);
```

```
mysql> DESC member;
```

Field	Type	Null	Key	Default	Extra
id	int(11)	NO	PRI	NULL	
name	varchar(64)	YES		NULL	
place	char(32)	YES		NULL	
age	int(11)	YES		NULL	

(b) 複数カラム

```
mysql> DESC member;
```

Field	Type	Null	Key	Default	Extra
id	int(11)	NO		NULL	
name	varchar(64)	YES		NULL	
place	char(32)	YES		NULL	
age	int(11)	YES		NULL	

```
ALTER TABLE member ADD PRIMARY KEY (id,name);
```

```
mysql> DESC member;
```

Field	Type	Null	Key	Default	Extra
id	int(11)	NO	PRI	NULL	
name	varchar(64)	NO	PRI	NULL	
place	char(32)	YES		NULL	
age	int(11)	YES		NULL	

(6) 主キー削除

削除前

Field	Type	Null	Key	Default	Extra
id	int(11)	NO	PRI	NULL	
name	varchar(64)	YES		NULL	
place	char(32)	YES		NULL	
age	int(11)	YES		NULL	

```
DROP INDEX `PRIMARY` ON member;
```

削除後

```
mysql> DESC member;
```

Field	Type	Null	Key	Default	Extra
id	int(11)	NO		NULL	
name	varchar(64)	YES		NULL	
place	char(32)	YES		NULL	
age	int(11)	YES		NULL	

6.8 JOIN

リレーショナルデータベースを活用するうえで、テーブルの関係をデータの抽出に反映させるための命令が JOIN である。この命令を確認するために、以下の表 26、表 27 について考える。

表 26 科目テーブル

科目(subject)	教員 ID(teacher_id)
電磁気学	404
電子回路	402
シミュレーション	404
ネットワーク基礎	407
オブジェクト指向	409
集積回路設計	408
計算機科学史	405
フーリエ解析	601
画像処理	401
ベクトル解析	603

表 27 教員テーブル

教員 ID(id)	教員(name)
401	押田 京一
402	荒井 善昭
403	楡井 雅巳
404	西村 治
405	藤澤 義範
406	大矢 健一
407	伊藤 祥一
408	芦田 和毅
409	藤田 悠

以下の表 26、表 27 を作成するための SQL は以下である。

```
CREATE DATABASE nnctj;
USE nnctj;
CREATE TABLE subject(subject VARCHAR (32), teacher_id INT);
CREATE TABLE teacher(id INT, name VARCHAR(32));
INSERT INTO subject (subject, teacher_id) VALUES
("電磁気学","404"),
("電子回路","402"),
("シミュレーション","404"),
("ネットワーク基礎","407"),
("オブジェクト指向","409"),
("集積回路設計","408"),
("計算機科学史","405"),
("フーリエ解析","601"),
("画像処理","401"),
("ベクトル解析","603");
INSERT INTO teacher (id, name) VALUES
("401","押田 京一"),
("402","荒井 善昭"),
("403","楡井 雅巳"),
("404","西村 治"),
("405","藤澤 義範"),
("406","大矢 健一"),
("407","伊藤 祥一"),
("408","芦田 和毅"),
("409","藤田 悠");
```

この SQL にて登録したデータベースのテーブルは以下のようにになっている。

```
mysql> SELECT * FROM subject;
```

subject	teacher_id
電磁気学	404
電子回路	402
シミュレーション	404
ネットワーク基礎	407
オブジェクト指向	409
集積回路設計	408
計算機科学史	405
フーリエ解析	601
画像処理	401
ベクトル解析	603

```
mysql> SELECT * FROM teacher;
```

id	name
401	押田 京一
402	荒井 善昭
403	楡井 雅巳
404	西村 治
405	藤澤 義範
406	大矢 健一
407	伊藤 祥一
408	芦田 和毅
409	藤田 悠

(1) 内部結合 (INNER JOIN)

```
table1 INNER JOIN table2 ON condition
```

例を以下に示す.

```
mysql> SELECT * FROM subject INNER JOIN teacher ON subject.teacher_id = teacher.id;
```

subject	teacher_id	id	name
画像処理	401	401	押田 京一
電子回路	402	402	荒井 善昭
電磁気学	404	404	西村 治
シミュレーション	404	404	西村 治
計算機科学史	405	405	藤澤 義範
ネットワーク基礎	407	407	伊藤 祥一
集積回路設計	408	408	芦田 和毅
オブジェクト指向	409	409	藤田 悠

繋がりがある要素のみ表示される. つながりがない, 「フーリエ変換」等は表示されていないことが分かる.

```
mysql> SELECT subject.subject, teacher.id, teacher.name FROM subject  
INNER JOIN teacher ON subject.teacher_id = teacher.id;
```

subject	id	name
画像処理	401	押田 京一
電子回路	402	荒井 善昭
電磁気学	404	西村 治
シミュレーション	404	西村 治
計算機科学史	405	藤澤 義範
ネットワーク基礎	407	伊藤 祥一
集積回路設計	408	芦田 和毅
オブジェクト指向	409	藤田 悠

(2) 外部結合 (OUTER JOIN)

(a) 左外部結合

左側のテーブルを主として、結合する.

```
table1 LEFT OUTER JOIN table2 ON condition
```

左側にあるテーブルのレコードはすべて表示されて、それに右側のテーブルが結合される.

```
mysql> SELECT * FROM subject LEFT OUTER JOIN teacher
      ON subject.teacher_id = teacher.id;
+-----+-----+-----+-----+
| subject          | teacher_id | id   | name      |
+-----+-----+-----+-----+
| 画像処理         | 401        | 401  | 押田 京一 |
| 電子回路         | 402        | 402  | 荒井 善昭 |
| 電磁気学         | 404        | 404  | 西村 治   |
| シミュレーション | 404        | 404  | 西村 治   |
| 計算機科学史     | 405        | 405  | 藤澤 義範 |
| ネットワーク基礎 | 407        | 407  | 伊藤 祥一 |
| 集積回路設計     | 408        | 408  | 芦田 和毅 |
| オブジェクト指向 | 409        | 409  | 藤田 悠   |
| フーリエ解析     | 601        | NULL | NULL      |
| ベクトル解析     | 603        | NULL | NULL      |
+-----+-----+-----+-----+
```

左側のレコードにない、結合の条件については、NULL として表示される.

(b) 右外部結合

```
table1 RIGHT OUTER JOIN table2 ON condition
```

右側のテーブルを主として、結合する.

```
mysql> SELECT * FROM subject RIGHT OUTER JOIN teacher
      ON subject.teacher_id = teacher.id;
+-----+-----+-----+-----+
| subject          | teacher_id | id   | name      |
+-----+-----+-----+-----+
| 電磁気学         | 404        | 404  | 西村 治   |
| 電子回路         | 402        | 402  | 荒井 善昭 |
| シミュレーション | 404        | 404  | 西村 治   |
| ネットワーク基礎 | 407        | 407  | 伊藤 祥一 |
| オブジェクト指向 | 409        | 409  | 藤田 悠   |
| 集積回路設計     | 408        | 408  | 芦田 和毅 |
| 計算機科学史     | 405        | 405  | 藤澤 義範 |
| 画像処理         | 401        | 401  | 押田 京一 |
| NULL            | NULL       | 403  | 楡井 雅巳 |
| NULL            | NULL       | 406  | 大矢 健一 |
+-----+-----+-----+-----+
```

7. ファイル

7.1 ディレクトリ設定

ファイルに実行結果を出力しようとする、標準では、以下のようにエラーとなる。

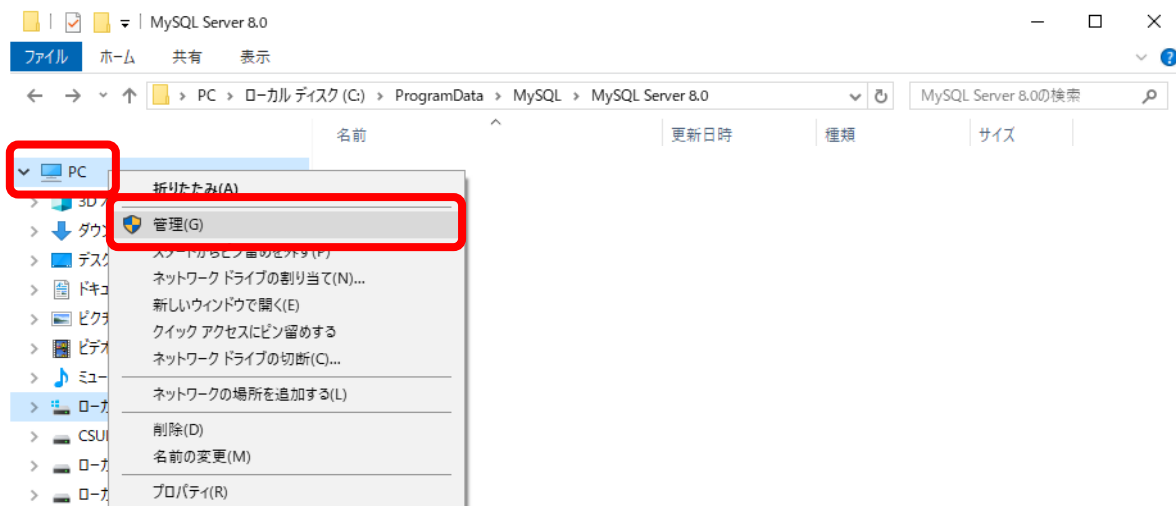
```
mysql> use tshirts_sales_db;
mysql> SELECT * FROM sales into outfile "test.txt";
ERROR 1290 (HY000): The MySQL server is running with the --secure-file-priv option
so it cannot execute this statement
```

「C:\ProgramData\MySQL\MySQL Server 8.4\my.ini」ファイル中を以下のように変更する。

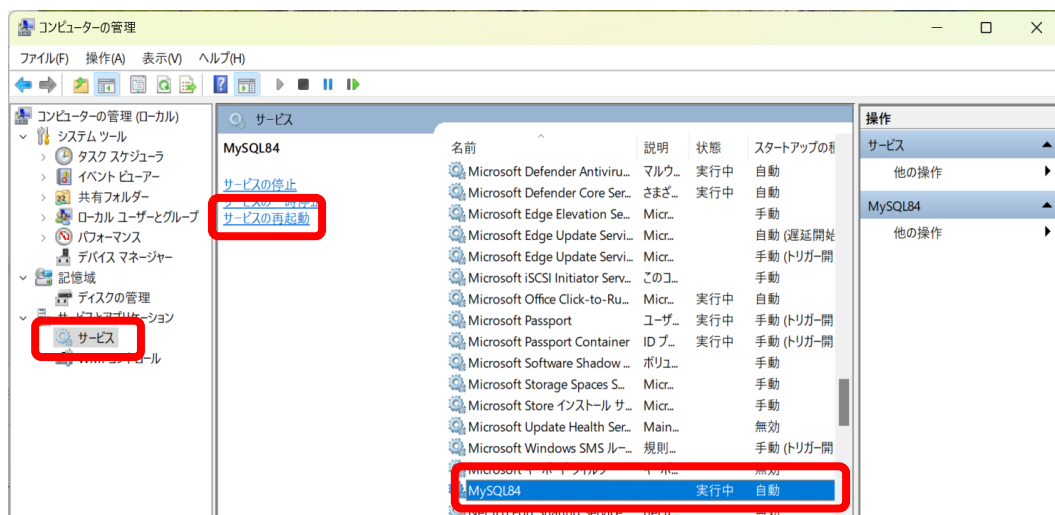
160 行目をコメントアウト、161 行目を追加

157	# This variable is used to limit the effect of data import and export operations, such as
158	# those performed by the LOAD DATA and SELECT ... INTO OUTFILE statements and the
159	# LOAD_FILE() function. These operations are permitted only to users who have the FILE privilege.
160	#secure-file-priv="C:/ProgramData/MySQL/MySQL Server 8.4/Uploads"
161	secure-file-priv=""

MySQL のサービスを再起動する。PC を右クリックして、「その他のオプション」から「管理」をクリックし、「コンピュータの管理」画面を開く。



「サービスとアプリケーション／サービス」にある「MySQL84」のサービスを選択し、「サービスの再起動」をクリックして、再起動する。



c ドライブの直下 (c:¥) に tmp フォルダを作成し、以下の SQL 文を実行する。

```
mysql> select * from sales into outfile 'c:/tmp/test.txt';
```

7.2 ファイルからの INSERT

(1) タブ区切り

実行前

```
mysql> use company;
mysql> select * from member;
+-----+-----+-----+-----+
| id | name      | place | age |
+-----+-----+-----+-----+
| 101 | 木村太郎  | 岩手   | 28  |
| 102 | 田中花子   | 宮城   | 38  |
| 103 | 中村裕子   | 宮城   | 41  |
| 104 | 中村康太   | 青森   | 41  |
+-----+-----+-----+-----+
```

文字コード: UTF-8, 区切り文字: タブ, 改行コード: LF とする, テキストファイルを作成する。

105	伊藤加奈子	福島	32
106	長谷川修次	秋田	45

SQL 文は以下ようになる。

```
LOAD DATA INFILE 'c:/tmp/member.txt' INTO TABLE member;
```

実行後

```
mysql> select * from member;
+-----+-----+-----+-----+
| id | name      | place | age |
+-----+-----+-----+-----+
| 101 | 木村太郎  | 岩手   | 28  |
| 102 | 田中花子   | 宮城   | 38  |
| 103 | 中村裕子   | 宮城   | 41  |
| 104 | 中村康太   | 青森   | 41  |
| 105 | 伊藤加奈子 | 福島   | 32  |
| 106 | 長谷川修次 | 秋田   | 45  |
+-----+-----+-----+-----+
```

(2) CSV

以下のような、CSV ファイルの場合は、区切り文字を指定する。

```
107,田村浩二,青森,24
```

```
108,上村優子,岩手,56
```

```
LOAD DATA INFILE 'c:/tmp/member2.txt' INTO TABLE member fields
terminated by ',';
```

実行後

```
mysql> select * from member;
```

id	name	place	age
101	木村太郎	岩手	28
102	田中花子	宮城	38
103	中村裕子	宮城	41
104	中村康太	青森	41
105	伊藤加奈子	福島	32
106	長谷川修次	秋田	45
107	田村浩二	青森	24
108	上村優子	岩手	56

7.3 コマンド読み込み

データベース作成や、SQL の操作などの命令が書かれたテキストファイルの内容を、SQL 文として入力するコマンドがある。

例えば、以下のように、データベース作成、データベース選択、テーブル作成、データ挿入、データ表示をする SQL 文をまとめる。

```
CREATE DATABASE sql12345;
USE sql12345;
CREATE TABLE books(
id INT NOT NULL PRIMARY KEY,
name varchar(128),
author char(64)
);
INSERT INTO books (id, name, author) VALUES
("10001","ライ麦畑でつかまえて","J・D・サリンジャー"),
("10002","モモ","ミヒヤエル・エンデ"),
("10003","二年間の休暇","ジュール ヴェルヌ")
;
SELECT id, name, author FROM books;
```

```
SOURCE c:/tmp/sql.txt
```

```
mysql> source c:/tmp/sql.txt
Query OK, 1 row affected (0.01 sec)

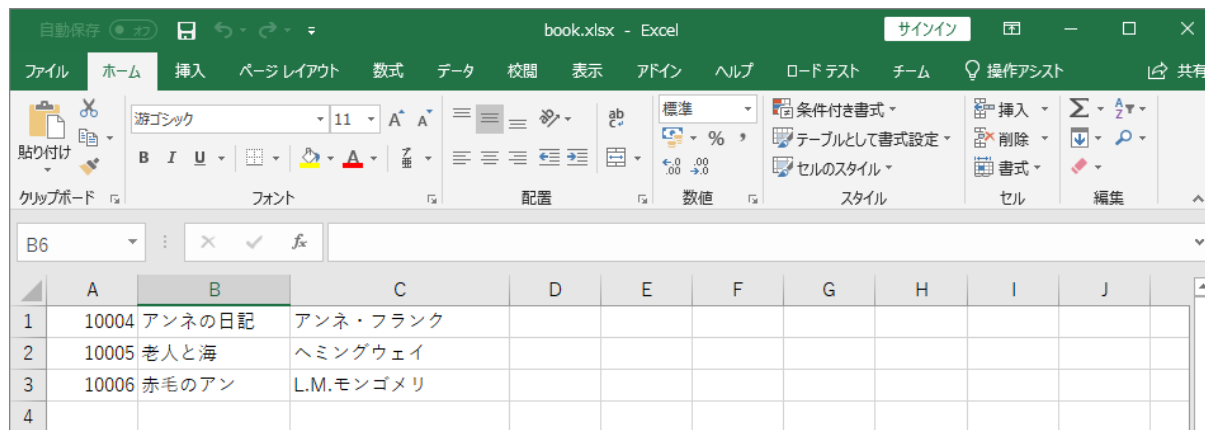
Database changed
Query OK, 0 rows affected (0.03 sec)

Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0
```

id	name	author
10001	ライ麦畑でつかまえて	J・D・サリンジャー
10002	モモ	ミヒヤエル・エンデ
10003	二年間の休暇	ジュール ヴェルヌ

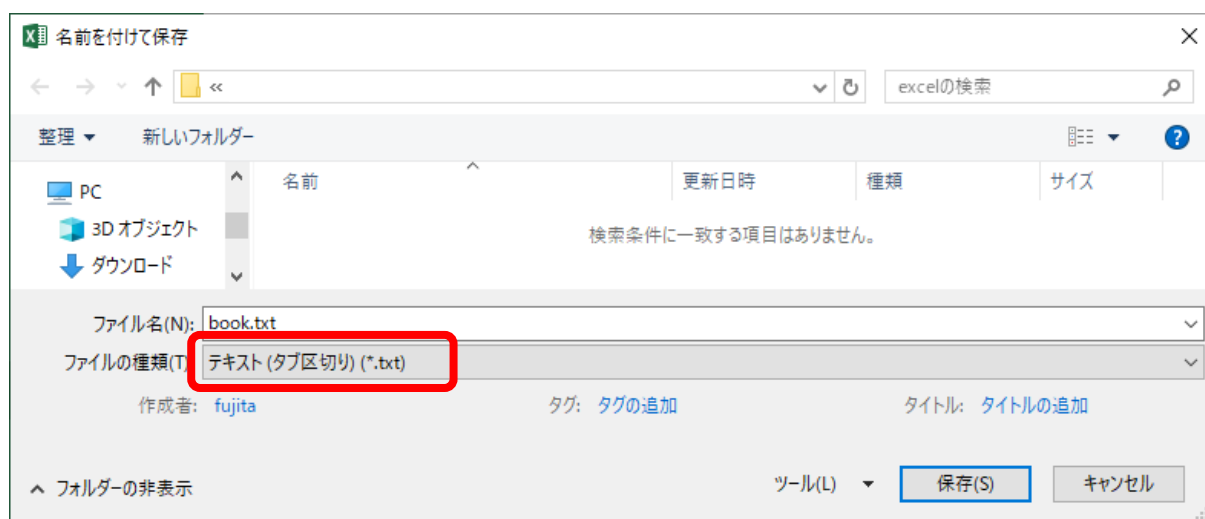
```
3 rows in set (0.00 sec)
```

7.4 Excel のデータ出力

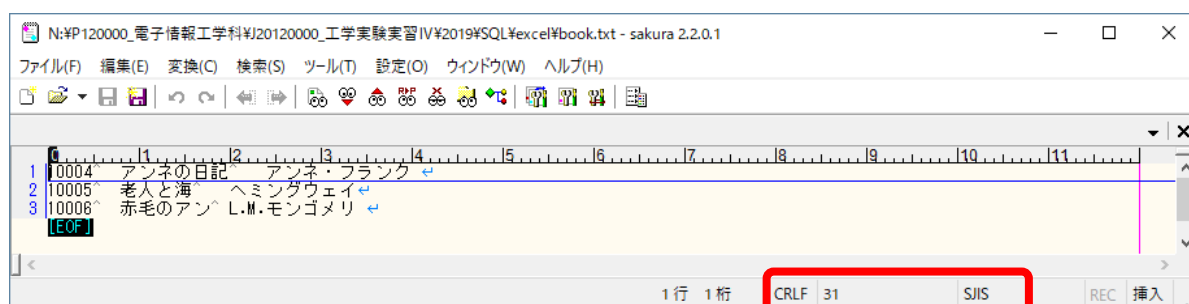


The screenshot shows the Microsoft Excel interface with a table containing book data. The table has columns A through J and rows 1 through 4. The data is as follows:

	A	B	C	D	E	F	G	H	I	J
1	10004	アンネの日記	アンネ・フランク							
2	10005	老人と海	ヘミングウェイ							
3	10006	赤毛のアン	L.M. モンゴメリ							
4										



The screenshot shows the '名前を付けて保存' (Save As) dialog box. The file name is 'book.txt'. The file type is 'テキスト(タブ区切り) (*.txt)', which is highlighted with a red rectangle. The dialog also shows the file location, file size, and a '保存(S)' (Save) button.



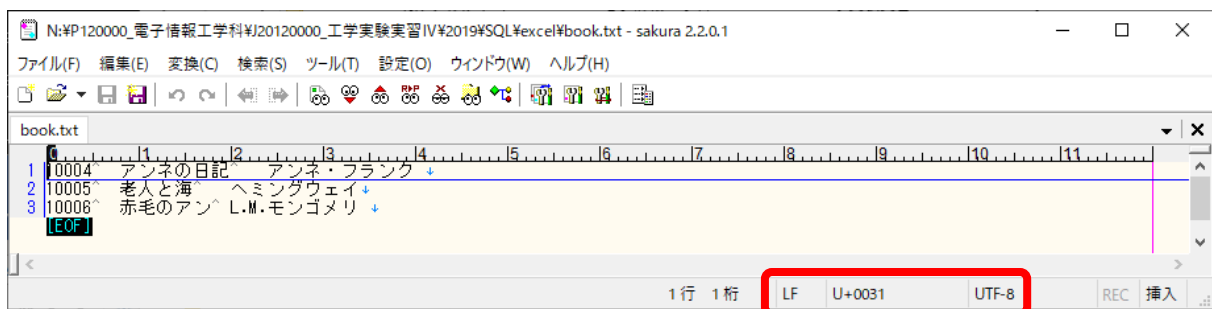
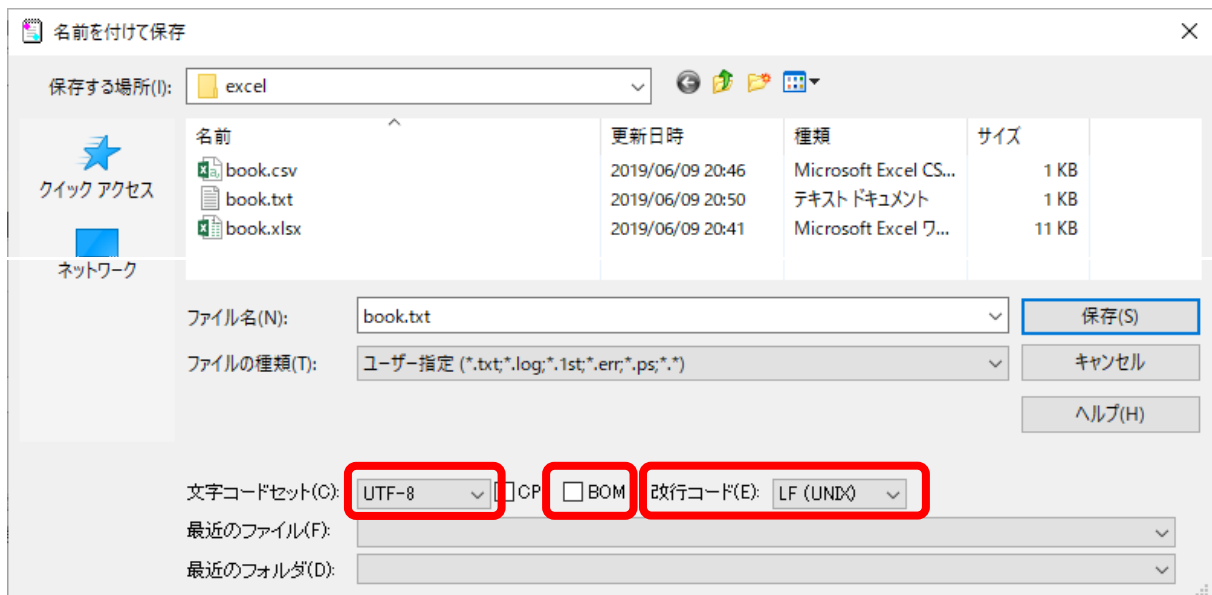
The screenshot shows a text editor window displaying the output of the Excel data. The text is as follows:

```

1 10004 アンネの日記 アンネ・フランク
2 10005 老人と海 ヘミングウェイ
3 10006 赤毛のアン L.M. モンゴメリ

```

The text is displayed in a monospaced font. The status bar at the bottom shows '1行 1桁' (1 line, 1 character) and 'CRLF 31' (CRLF, 31 characters), which is highlighted with a red rectangle.



出力↓ファイルをレコードとして読み込むには、以下ようになる。

```
mysql> LOAD DATA INFILE 'c:/tmp/book.txt' INTO TABLE books;
```

```
mysql> select * from books;
```

id	name	author
10001	ライ麦畑でつかまえて	J・D・サリンジャー
10002	モモ	ミヒヤエル・エンデ
10003	二年間の休暇	ジュール ヴェルヌ
10004	アンネの日記	アンネ・フランク
10005	老人と海	ヘミングウェイ
10006	赤毛のアン	L. M. モンゴメリ

8. インストール

MySQL を用いる.

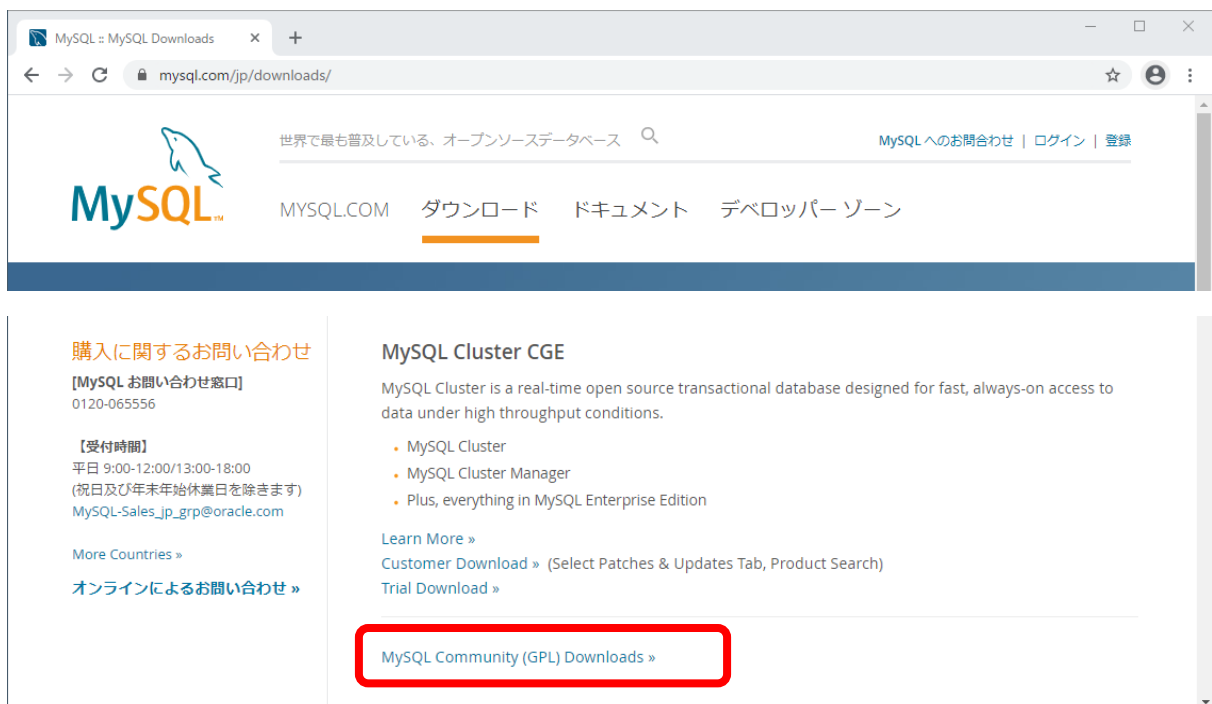
8.1 ダウンロード

(1) 以下のアドレスをウェブブラウザで開く

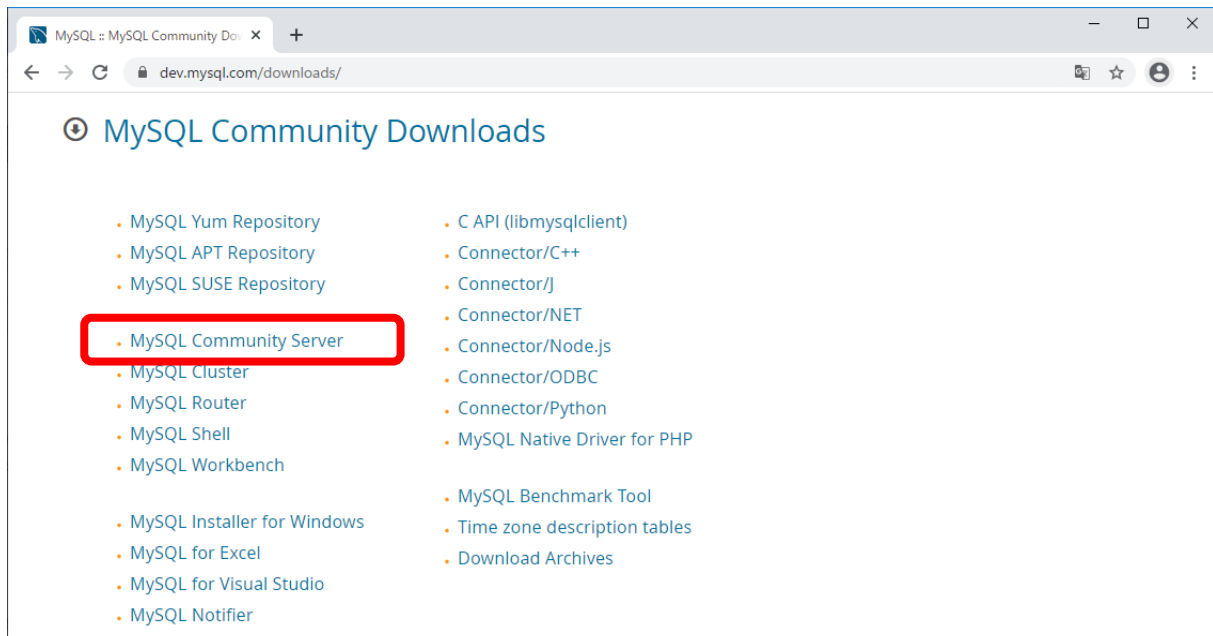
<https://www.mysql.com/jp/>



(2) 「ダウンロード」メニューを選ぶ

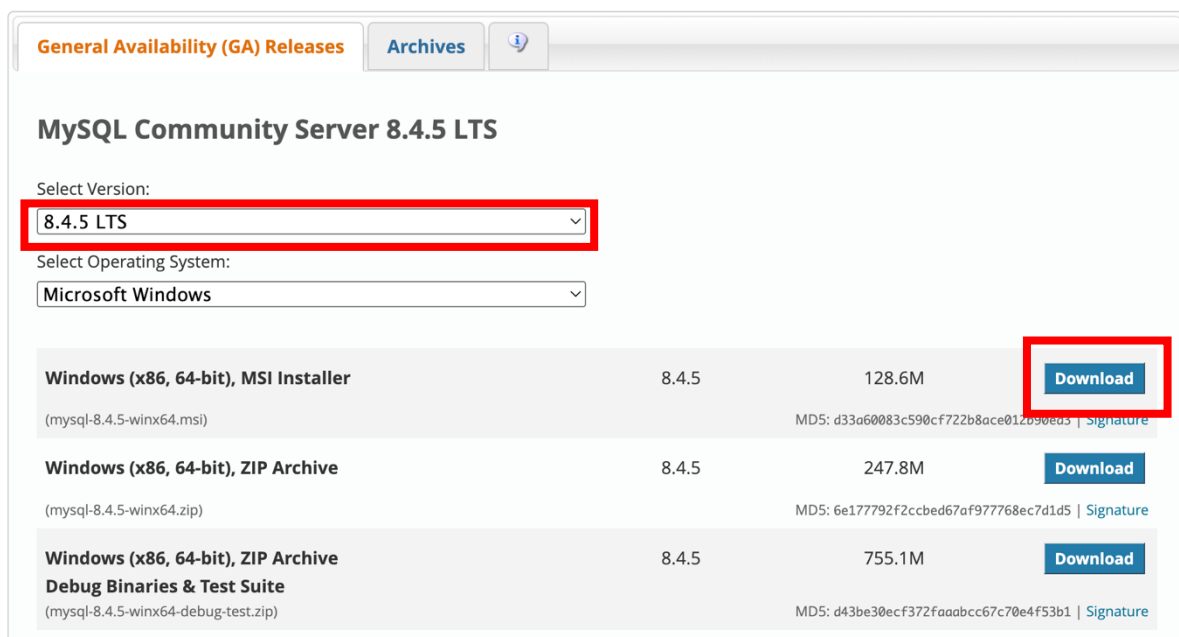


(3) 「MySQL Community Server」をクリックする



(4) Select Version は「X.X.X LTS」を選択 (最新バージョンを選択)

(5) Windows(x86, 64bit), MSI Installer の方をダウンロード



(6) No thanks, just start my download

MySQL Community Downloads

Login Now or Sign Up for a free account.

An Oracle Web Account provides you with the following advantages:

- Fast access to MySQL software downloads
- Download technical White Papers and Presentations
- Post messages in the MySQL Discussion Forums
- Report and track bugs in the MySQL bug system

Login »

using my Oracle Web account

Sign Up »

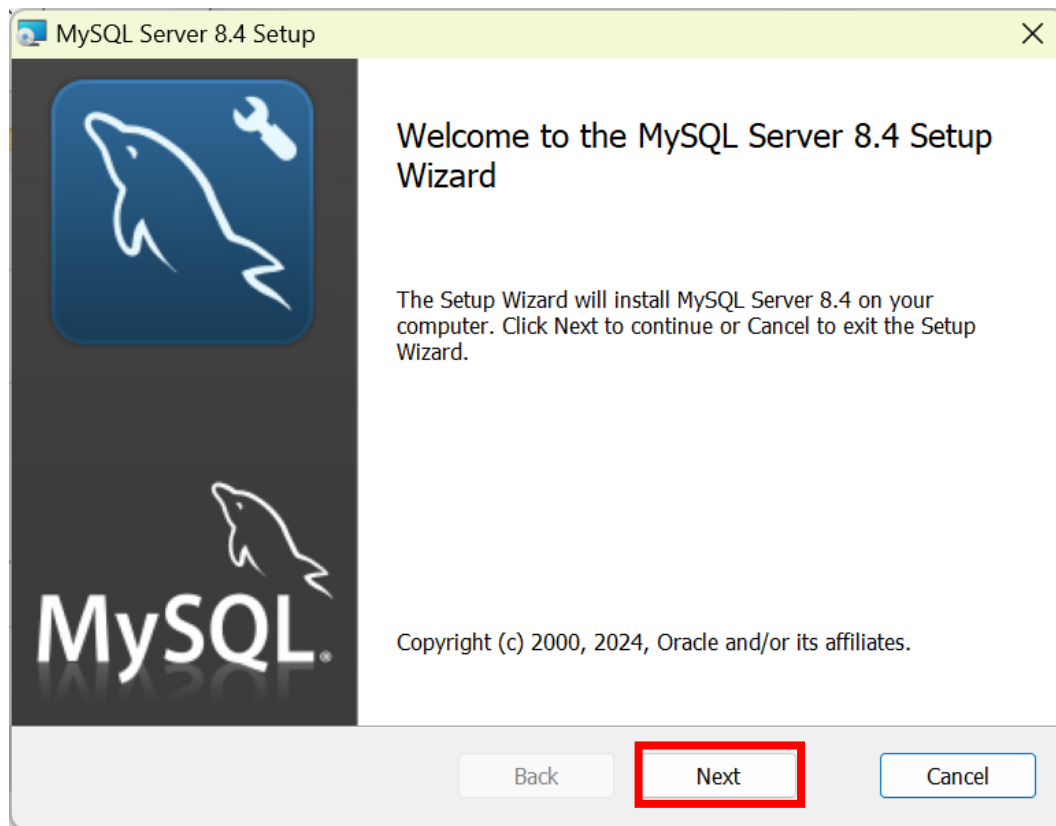
for an Oracle Web account

MySQL.com is using Oracle SSO for authentication. If you already have an Oracle Web account, click the Login link. Otherwise, you can signup for a free account by clicking the Sign Up link and following the instructions.

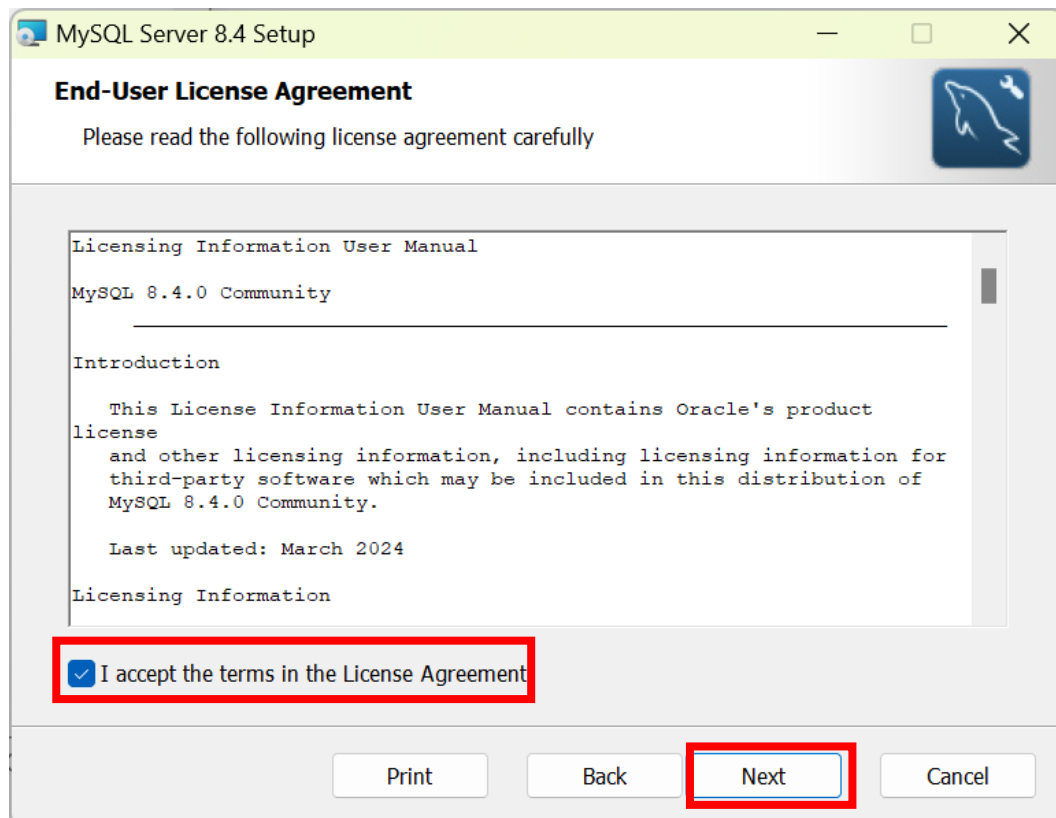
No thanks, just start my download.

8.2 インストール開始

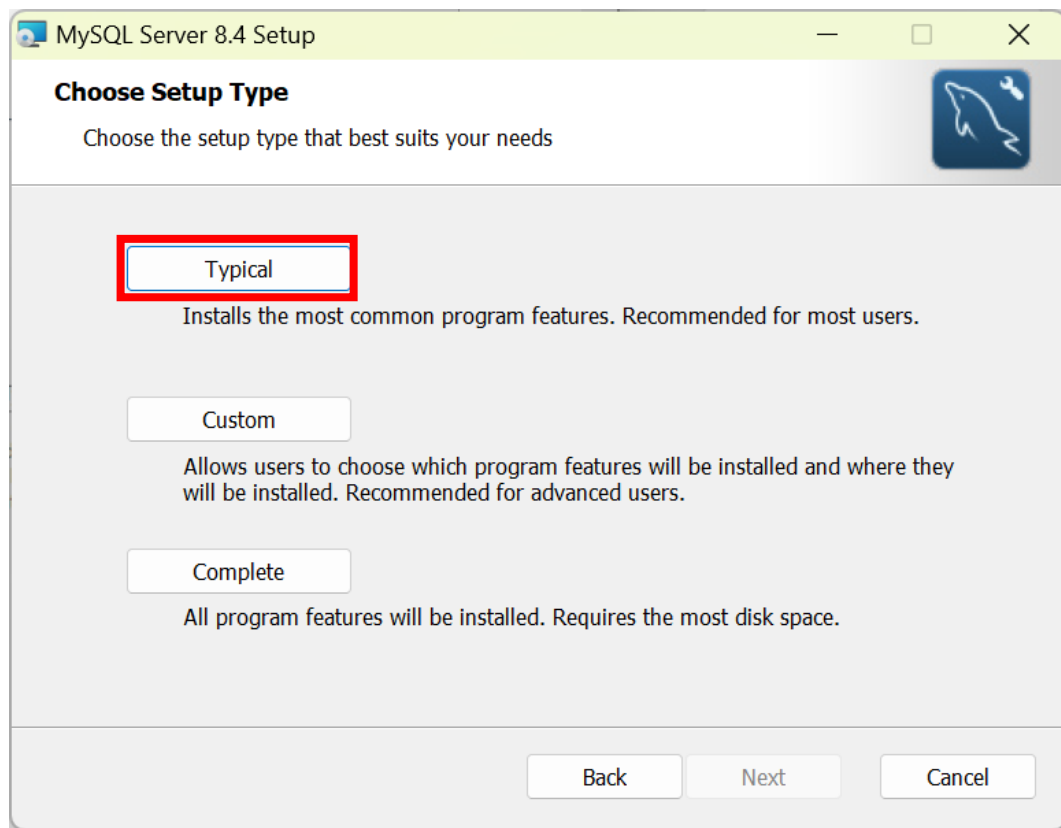
ダウンロードしたファイル「mysql-8.4.5-winx64.msi」をダブルクリックして、インストールを開始する。



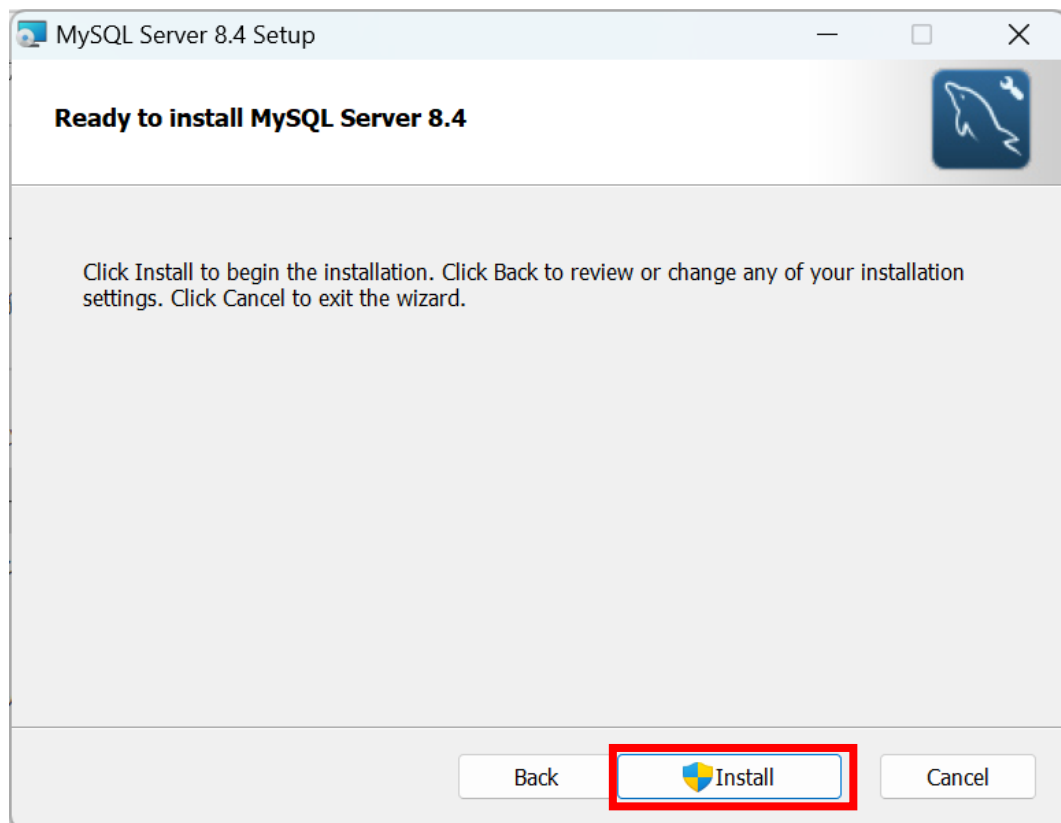
(1) 「I accept the terms in the License Agreement」にチェックを入れる



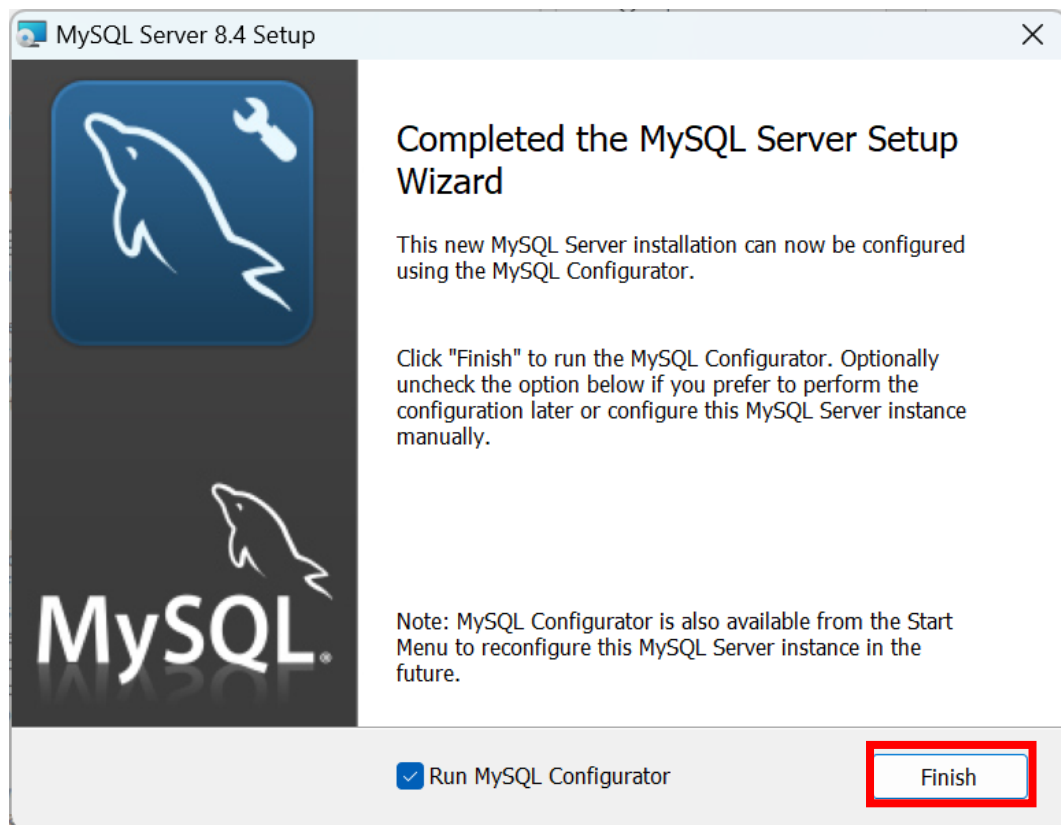
(2) 「Typical」 を選択



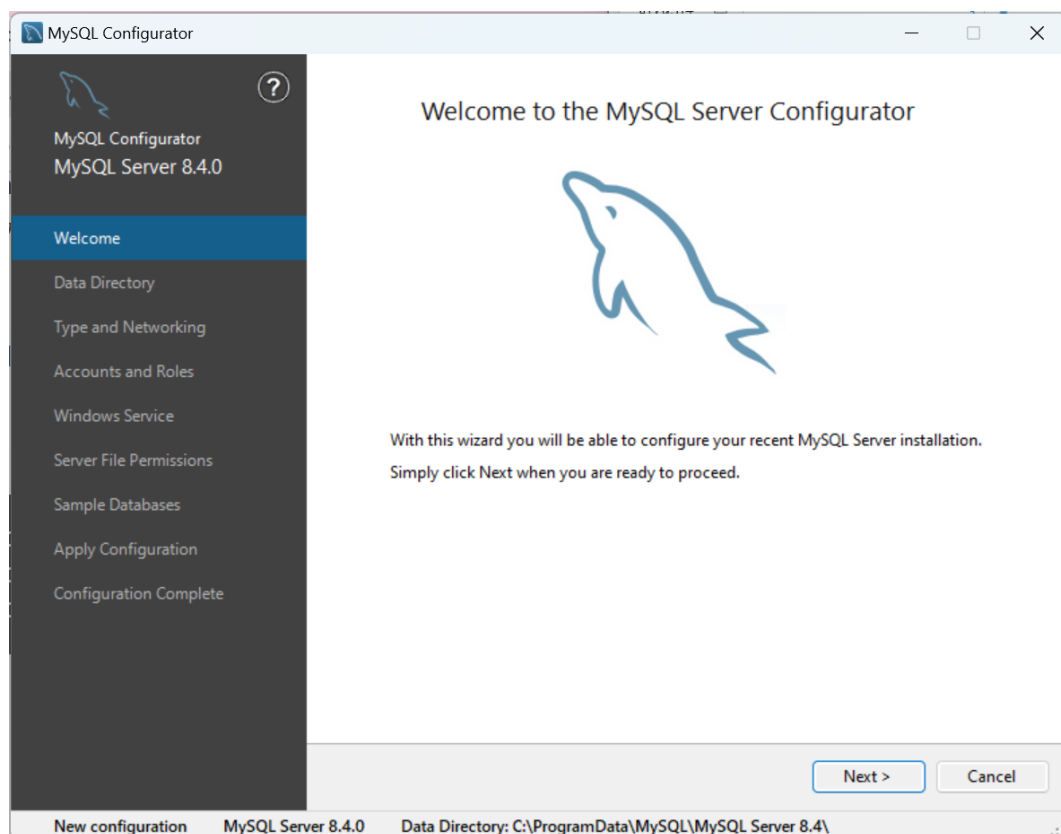
(3) 「Install」 をクリック



「Finish」をクリック

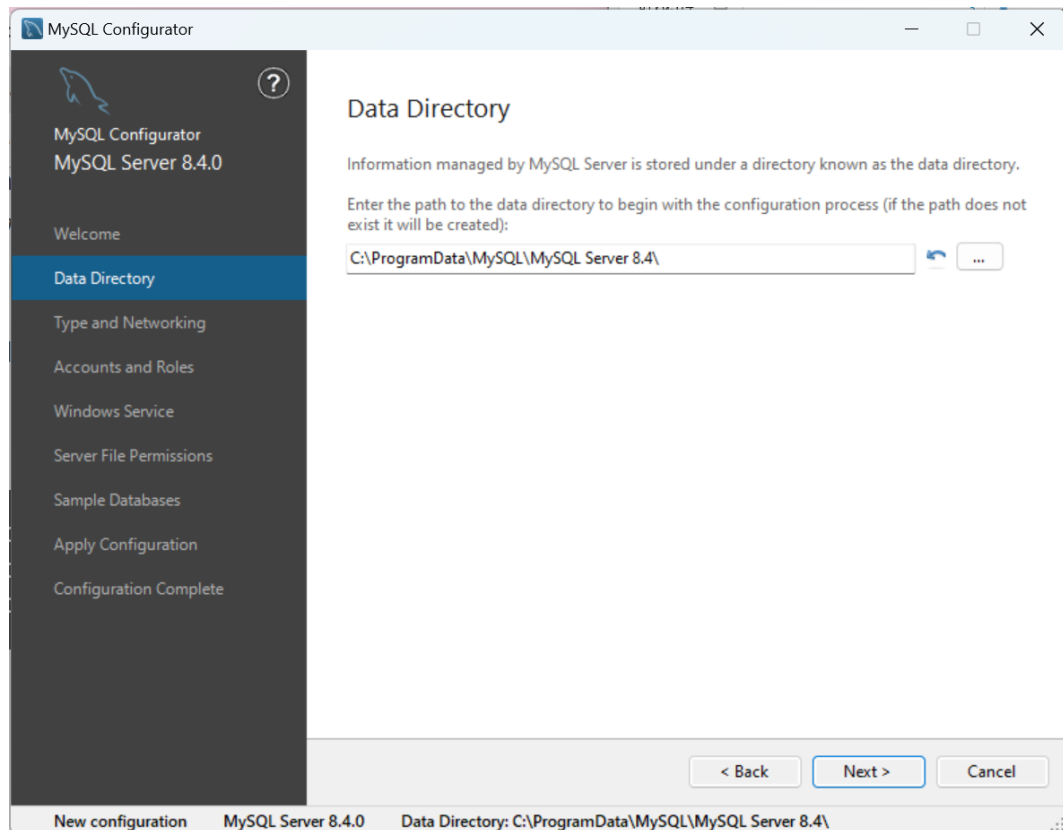


(4) 「MySQL Configurator」が立ち上がる

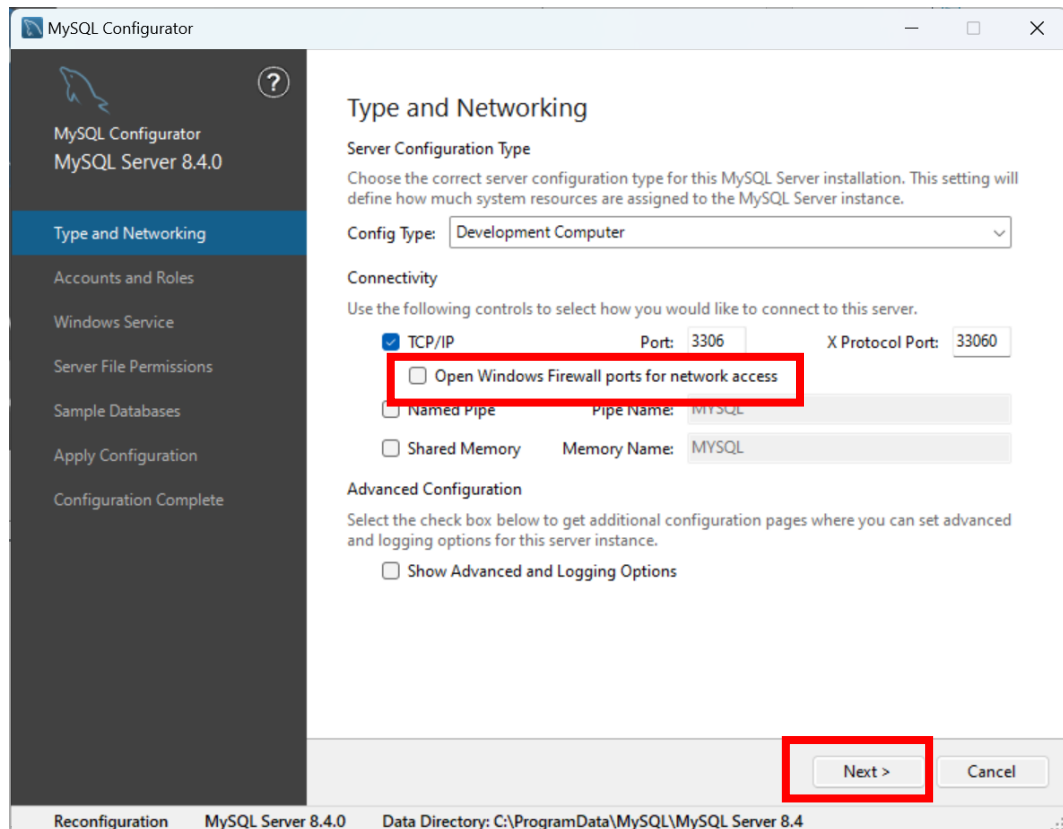


(5) ディレクトリの指定

デフォルトのままでよければ、そのまま「Next」をクリック。

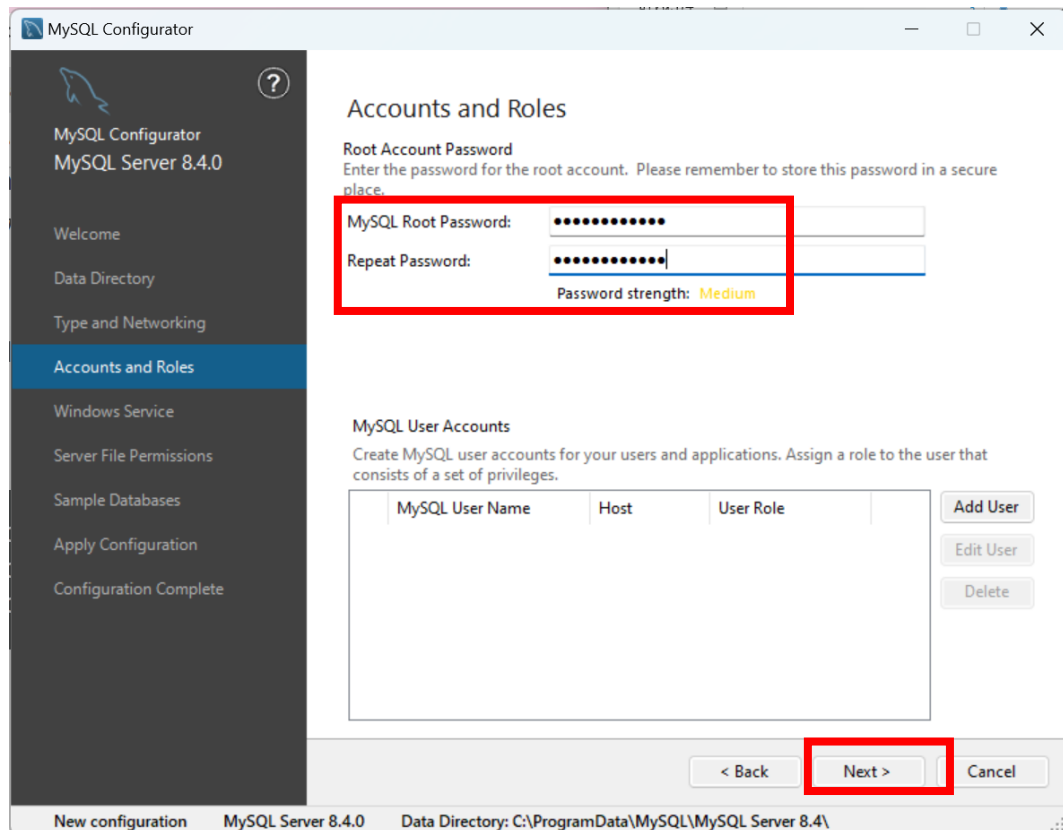


(6) 「Open Windows Firewall ports for network access」のチェックを外す



(7) パスワードを決める

毎回入力を求められるので忘れないように注意.



The screenshot shows the 'Accounts and Roles' configuration window in MySQL Configurator. The left sidebar lists various configuration steps, with 'Accounts and Roles' currently selected. The main area is titled 'Accounts and Roles' and contains a section for 'Root Account Password'. This section has two input fields: 'MySQL Root Password:' and 'Repeat Password:', both filled with masked characters. A red rectangle highlights these two fields. Below them, the 'Password strength' is indicated as 'Medium'. To the right of the password fields is a 'MySQL User Accounts' section with a table for adding users. At the bottom right, the 'Next >' button is highlighted with a red rectangle. The status bar at the bottom shows 'New configuration', 'MySQL Server 8.4.0', and the data directory path.

MySQL Configurator
MySQL Server 8.4.0

Welcome
Data Directory
Type and Networking
Accounts and Roles
Windows Service
Server File Permissions
Sample Databases
Apply Configuration
Configuration Complete

Accounts and Roles

Root Account Password
Enter the password for the root account. Please remember to store this password in a secure place.

MySQL Root Password: [masked]
Repeat Password: [masked]
Password strength: Medium

MySQL User Accounts

Create MySQL user accounts for your users and applications. Assign a role to the user that consists of a set of privileges.

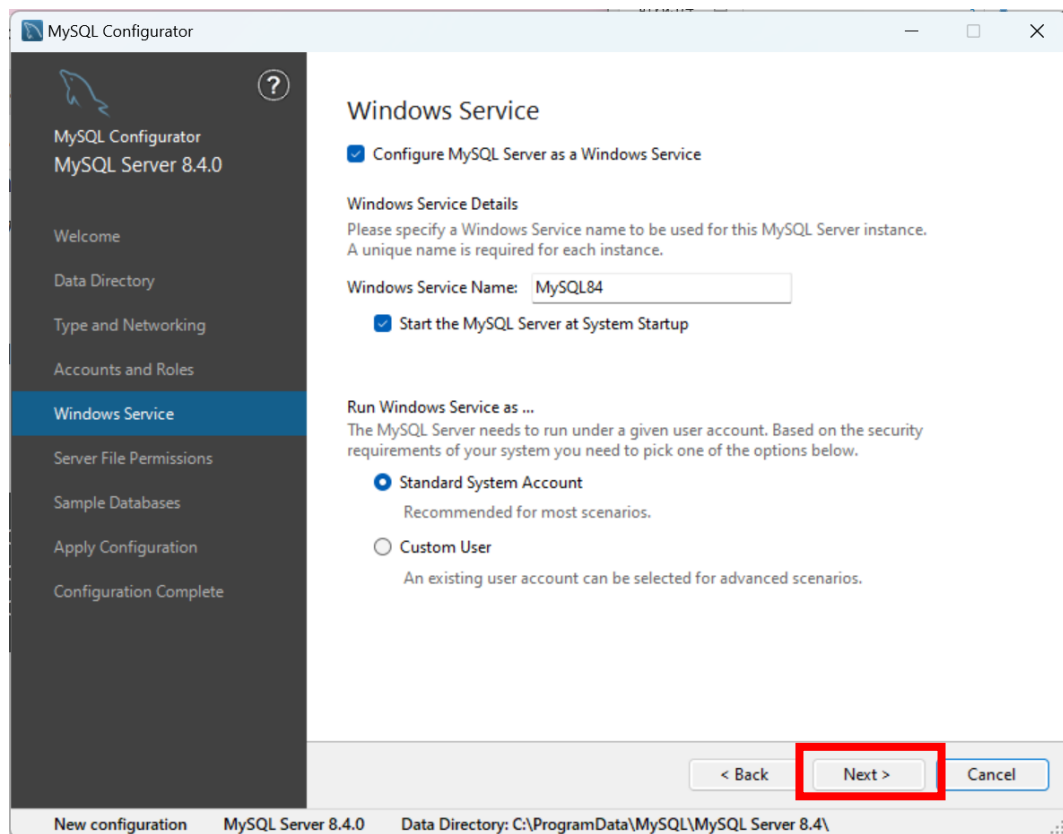
MySQL User Name	Host	User Role
-----------------	------	-----------

Add User
Edit User
Delete

< Back **Next >** Cancel

New configuration MySQL Server 8.4.0 Data Directory: C:\ProgramData\MySQL\MySQL Server 8.4\

「Next」をクリック.



The screenshot shows the 'Windows Service' configuration window in MySQL Configurator. The left sidebar is the same as the previous screen, but 'Windows Service' is now selected. The main area is titled 'Windows Service' and shows that the option 'Configure MySQL Server as a Windows Service' is checked. Below this, the 'Windows Service Details' section asks for a service name, with 'MySQL84' entered. The 'Start the MySQL Server at System Startup' checkbox is also checked. The 'Run Windows Service as ...' section shows 'Standard System Account' selected. At the bottom right, the 'Next >' button is highlighted with a red rectangle. The status bar at the bottom is identical to the previous screen.

MySQL Configurator
MySQL Server 8.4.0

Welcome
Data Directory
Type and Networking
Accounts and Roles
Windows Service
Server File Permissions
Sample Databases
Apply Configuration
Configuration Complete

Windows Service

☒ Configure MySQL Server as a Windows Service

Windows Service Details

Please specify a Windows Service name to be used for this MySQL Server instance. A unique name is required for each instance.

Windows Service Name: MySQL84

☒ Start the MySQL Server at System Startup

Run Windows Service as ...

The MySQL Server needs to run under a given user account. Based on the security requirements of your system you need to pick one of the options below.

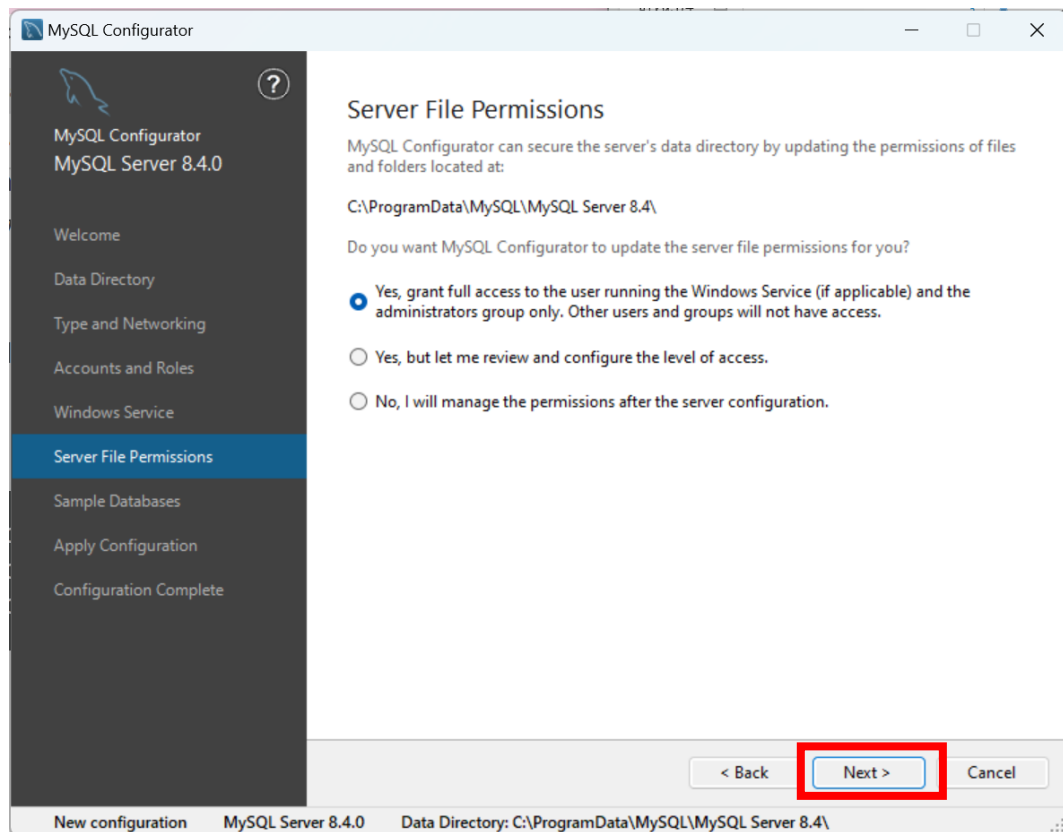
☒ Standard System Account
Recommended for most scenarios.

☐ Custom User
An existing user account can be selected for advanced scenarios.

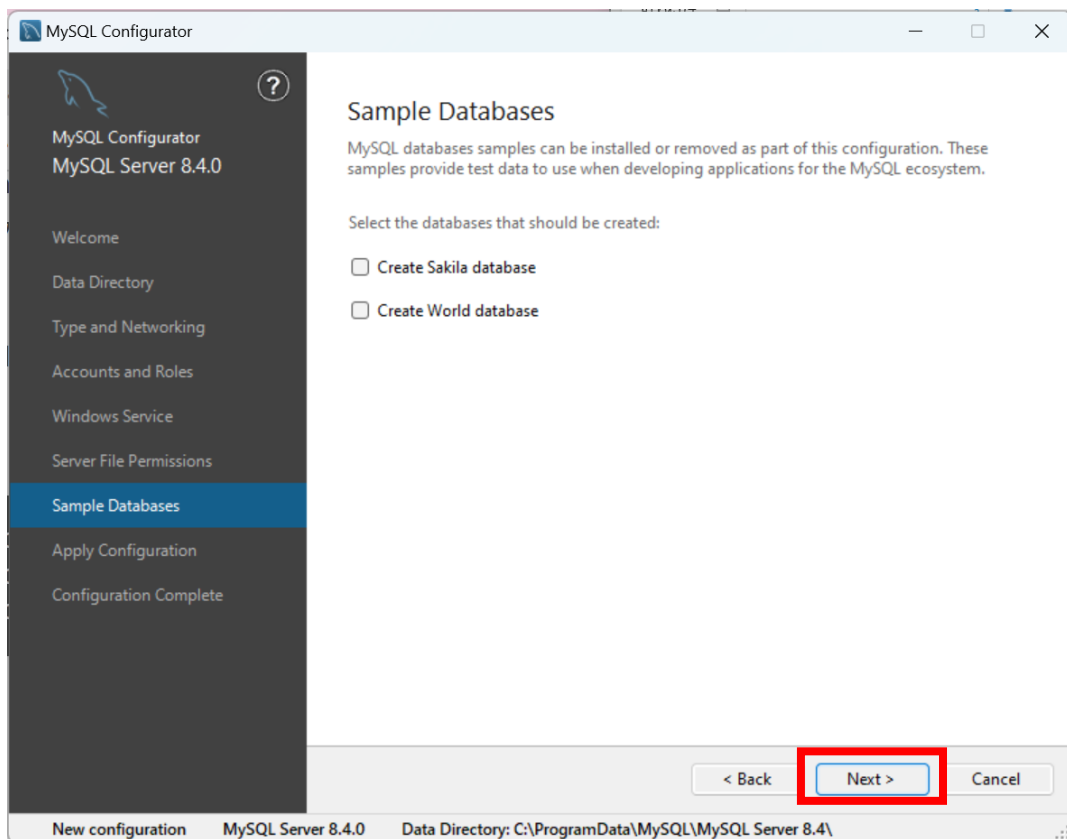
< Back **Next >** Cancel

New configuration MySQL Server 8.4.0 Data Directory: C:\ProgramData\MySQL\MySQL Server 8.4\

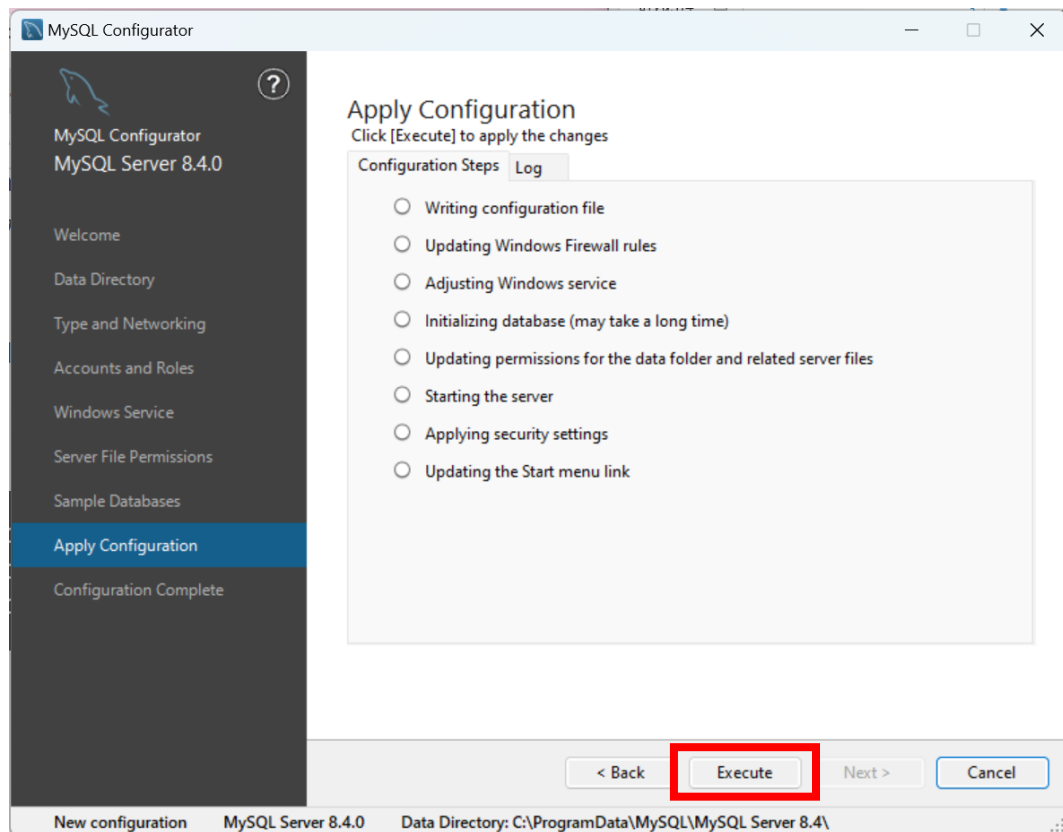
「Next」をクリック。



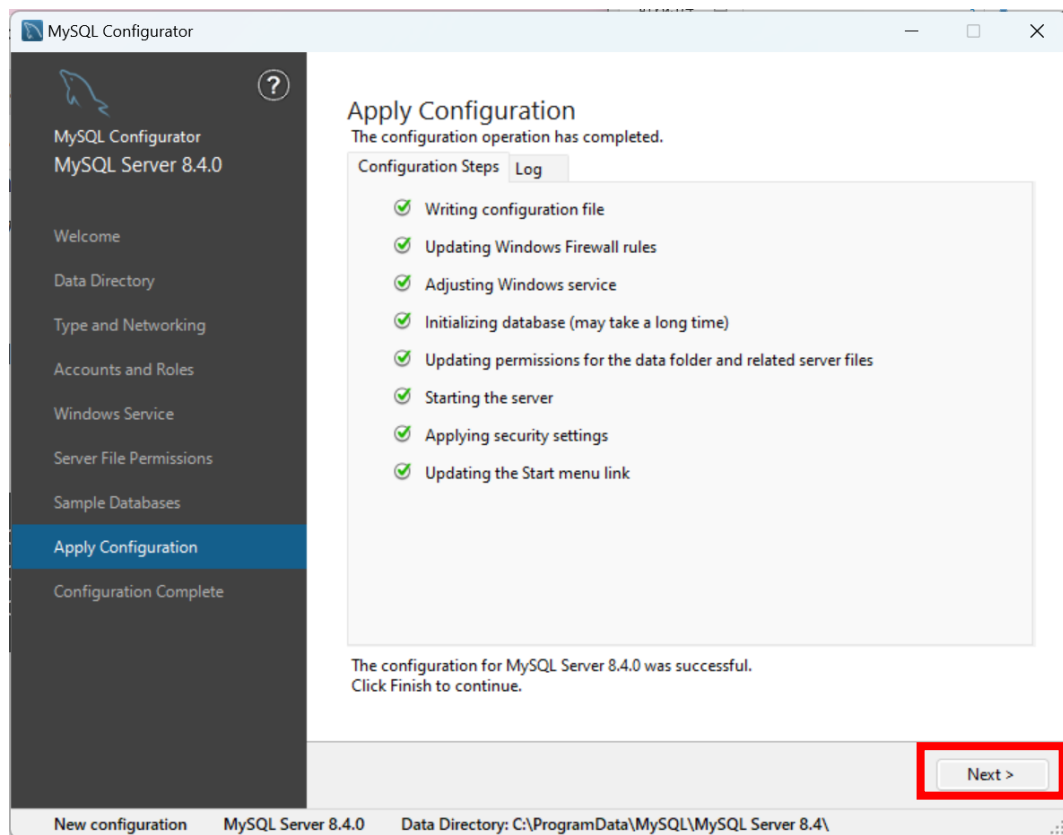
「Next」をクリック。



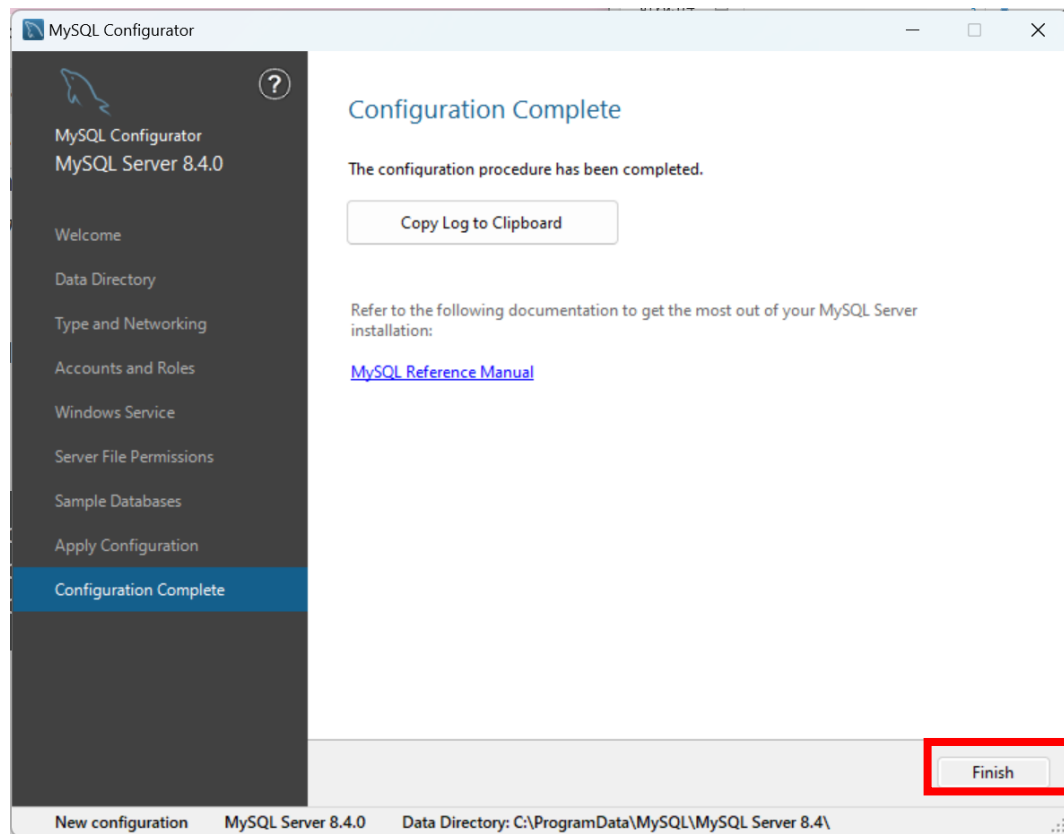
(8) 「Execute」をクリックしてしばらく待つ



「Next」をクリック.



(9) 「Finish」 をクリックしてインストール完了

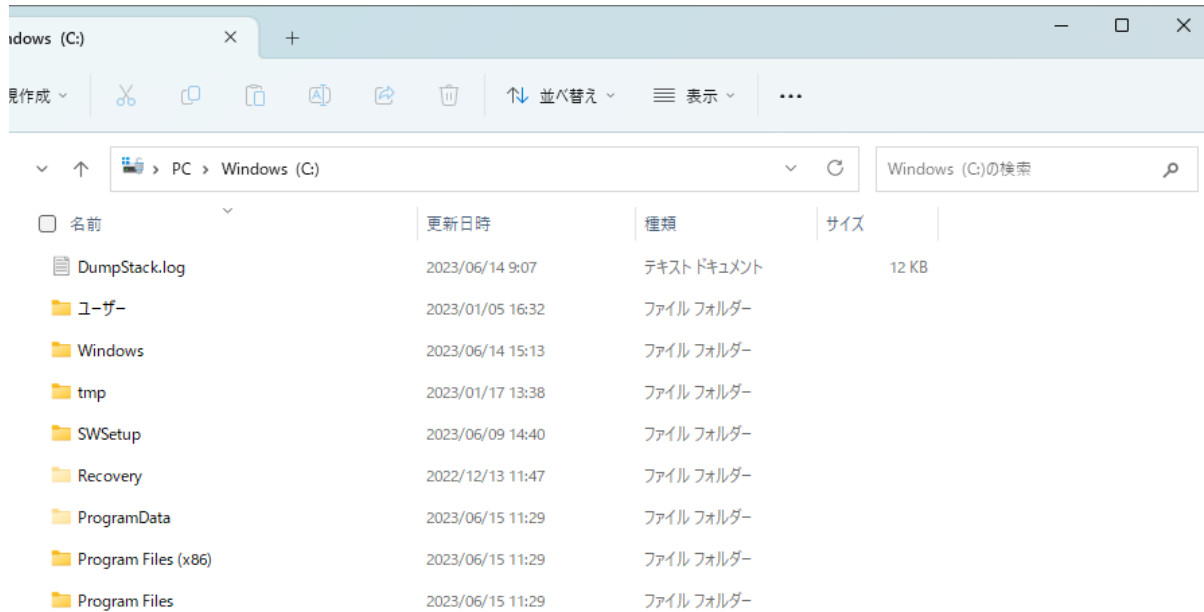


8.3 設定 (Unicode 版を使用する場合は不要)

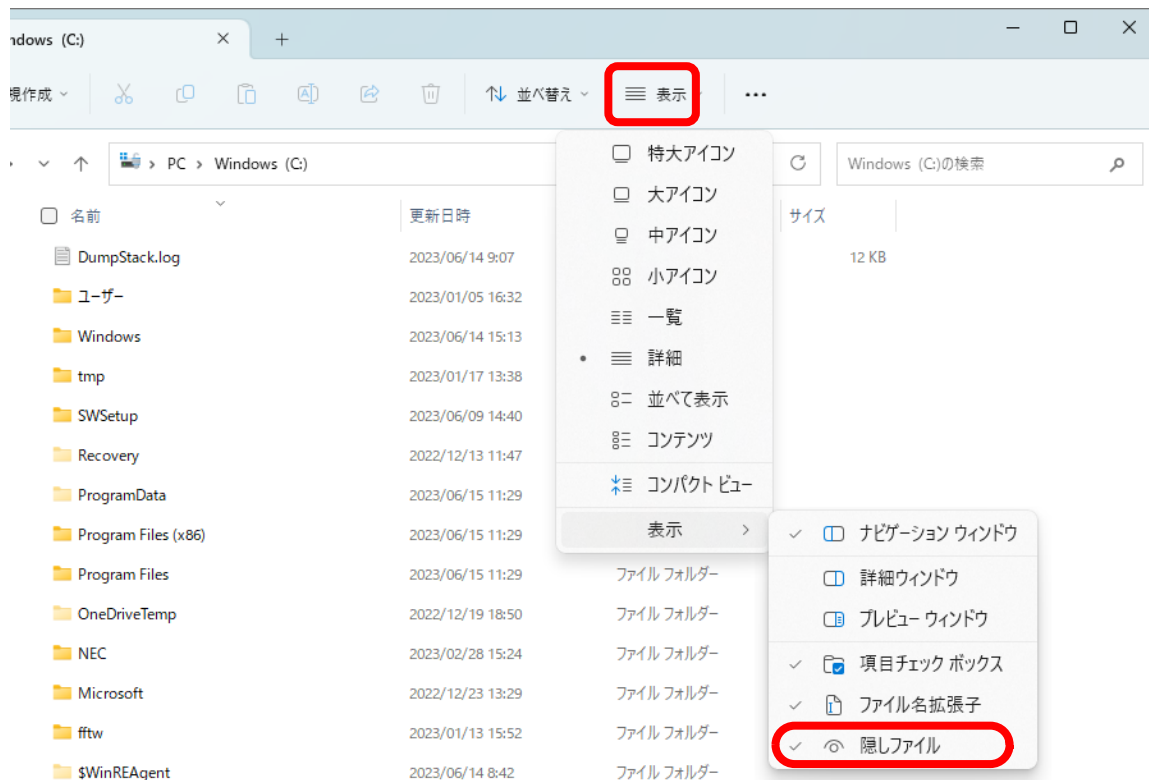
本実験では Unicode 版を使用するため、この設定は不要であるが、参考として示す。

まずは言語の設定をする。

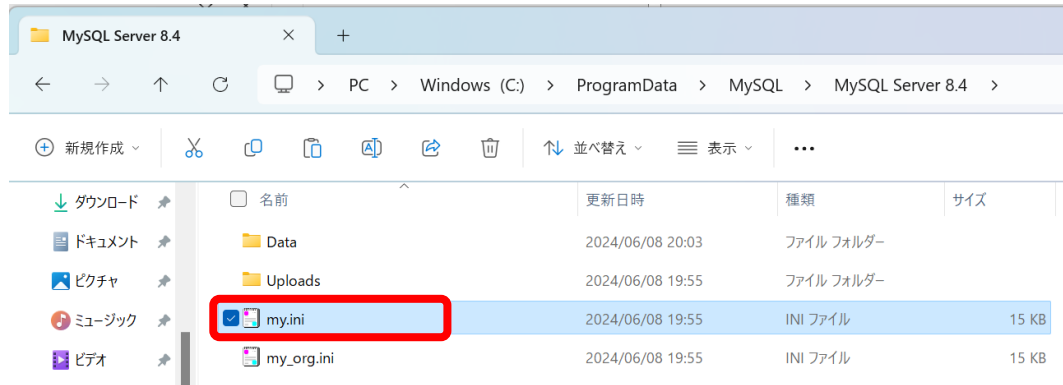
(1) 設定ファイルを開く



ProgramData フォルダは、隠しファイルであるので、表示されない設定になっている場合には、アクセスできない。その時には、「表示」メニューから「隠しファイル」にチェックを入れて、表示させるように設定する。



my.ini ファイルが、MySQL の設定ファイルであるためこれを編集する。ファイルを編集する前に、「C:\ProgramData\MySQL\MySQL Server 8.0\my_org.ini」などとして、ファイルをコピーして、バックアップしておくこと。また、設定ファイルを編集するときには、Windows のメモ帳は使わない。文字コードや改行コードを適切に設定できるテキストエディタを用いること。

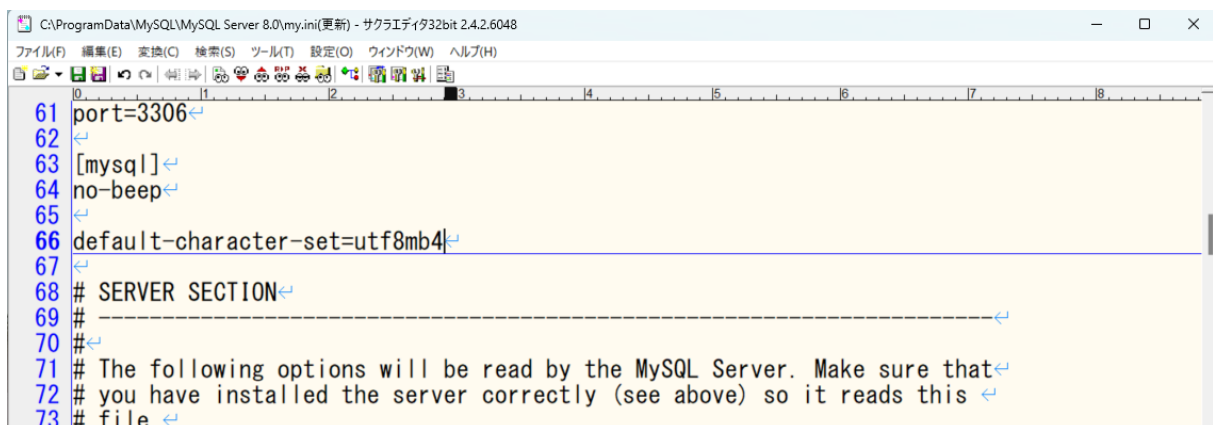


(2) 設定する

編集できない場合は、管理者権限でエディタを起動してから my.ini ファイルを開く。

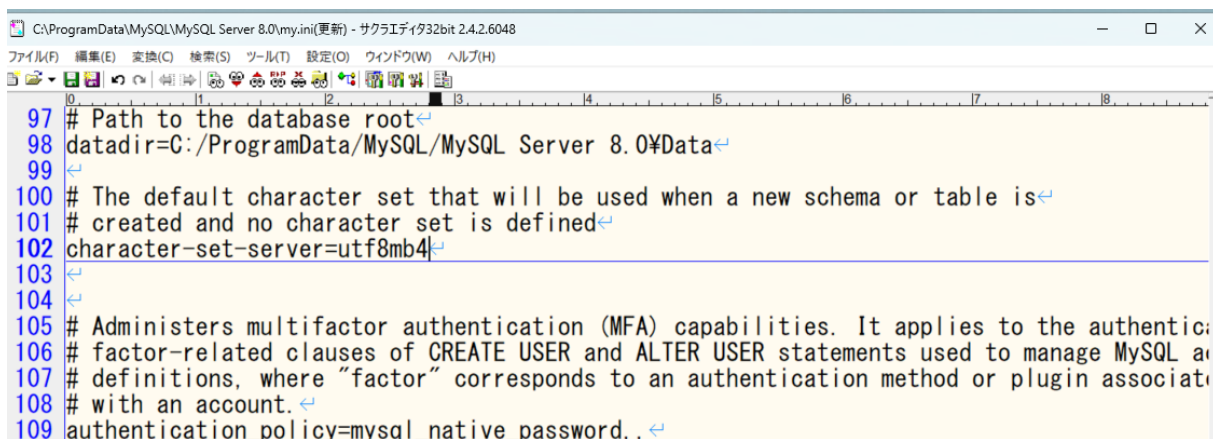
66 行目、#を外して以下のようにする。

```
default-character-set=utf8mb4
```



101 行目、#を外して以下のようにする。

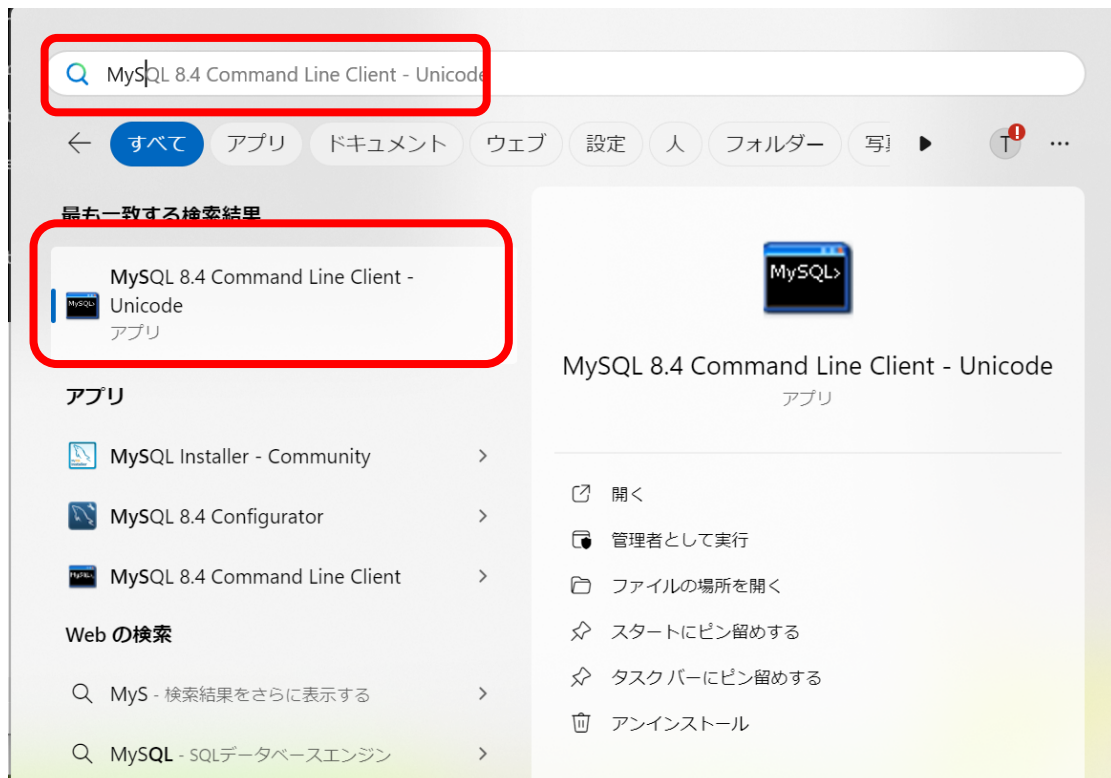
```
character-set-server=utf8mb4
```



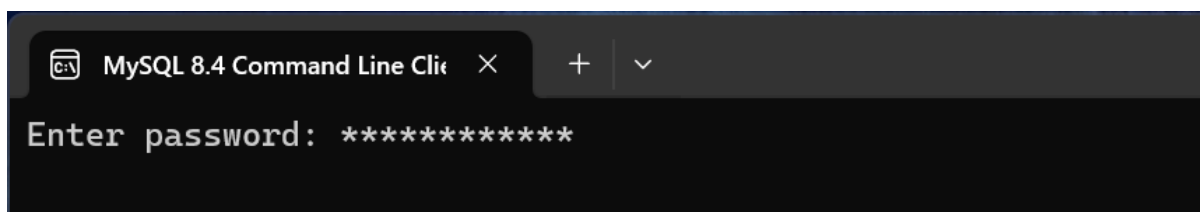
8.4 実行

コマンドプロンプトや PowerShell などから実行することもできるが、パスを通す必要があるので、ここでは、MySQL のインストールで用意される「Command Line Client Unicode」を使う。

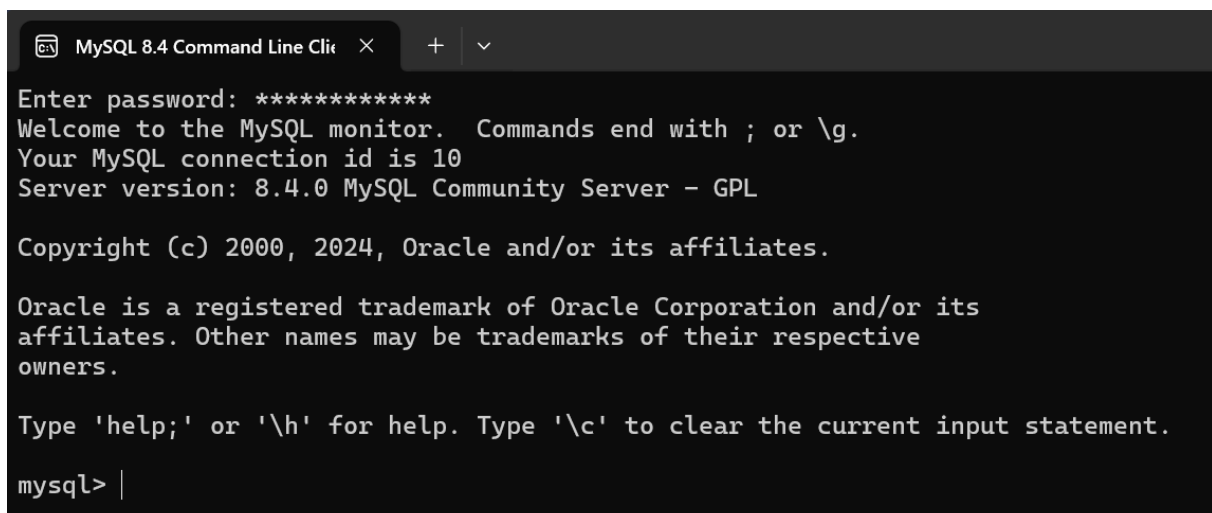
(1) スタートメニューから「MySQL」と入力する



(2) パスワードを入力する



(3) SQL のコマンドラインツールが立ち上がる



9. 課題

9.1 課題 1

表 28 のテーブルを正規化（第三正規化まで）せよ。また、第三正規化した結果を ER 図で表せ。ただし、第一正規化から第二正規化、第三正規化に至る途中経過についても報告せよ。第三正規化では、外部キー（他のテーブルと関連づけるカラム）の設定まで行うこと。

学生成績表のデータについては、以下の点に注意すること。

学籍番号は、1 人につき 1 つ付与されており、全て異なる値である。例えば、山川一郎が 2 人（同姓同名の別人）いるとしたら、それぞれに異なる学籍番号が付与される。

教員コードも学籍番号と同様に、1 人につき 1 つ付与されており、全て異なる値である。

同じ科目名の科目は、複数存在しない。

1 つの科目を担当する教員は必ず 1 名とする。

成績の「入力日」は成績を入力した日付とする。

表 28 学生成績表

学籍番号	学生名	年齢	性別	系	校舎	科目名	点数	判定	教員コード	教員名	入力日
26401	山川 一郎	18	男	都市デザイン系	C 棟	国語	65	可	G002	長松 次郎	2025-06-10
						物理	42	不可	G001	長松 遥	2025-06-11
						現代社会	79	良	G005	高田 真由	2025-06-14
26205	谷泉 太郎	19	男	情報エレクトロニクス系	B 棟	物理	76	良	G001	長松 遥	2025-06-11
						電気工学	66	可	E012	三浦 裕	2025-06-12
						信号処理	90	優	E012	三浦 裕	2025-06-12
26132	海土 花子	20	女	機械ロボティクス系	A 棟	物理	87	優	G001	長松 遥	2025-06-10
						機械力学	78	良	M301	上条 雄三	2025-06-11
						計測工学	100	優	M553	西岡 剛	2025-06-12

表 29 点数と判定

点数	判定
0～59	不可
60～69	可
70～79	良
80～100	優

注意 1：点数と判定は表 29 のように対応させること．第三正規化まで完了すると，テーブル表 29 を合わせて全部で 8 個となる．

注意 2：表 29 は，以下のように点数の上限，下限が定義されたテーブルになっていると考える．このテーブルは他のテーブルとリレーションを持たない．

id	上限	下限	判定
1	59	0	不可
2	?	?	可
3	?	?	良
4	?	?	優

9.2 課題 2

課題 1 で第三正規化したテーブルを表 30 に示す設定を用いて作成せよ．定義したデータベース，テーブルを生成する SQL 文と，データを挿入する SQL 文を定義する．テーブルを生成する SQL 文は，課題 1 で作成した ER 図から SQL エクスポートにより取得せよ．また，テーブルを実装した結果として，SELECT 文を実行し，各テーブルに登録した内容を表示せよ．

表 30 カラムの設定

項目	カラム（論理名）	カラム（物理名）	型	必須／非必須
学籍番号	学籍番号	code	CHAR(5)	必須
学生名	名前	name	VARCHAR(40)	必須
年齢	年齢	age	INT	非必須
性別	性別	gender	CHAR(4)	非必須
系	系	course	VARCHAR(40)	必須
校舎	校舎	place	VARCHAR(40)	必須
科目名	科目名	subject	VARCHAR(40)	必須
教員コード	教員コード	teacher_id	CHAR(4)	必須
教員名	名前	name	VARCHAR(40)	必須
点数	点数	score	INT	必須
判定	判定	result	VARCHAR(2)	必須
入力日	入力日	date	DATE	必須

9.3 課題 3

課題 2 で登録したデータ（第三正規化したテーブル）に対し、以下の問いに答えよ。レポートには、SQL 文も実行結果も両方報告すること。

- 1) 第 1 正規化したときの表を表示する SQL を定義せよ。
- 2) いずれかの科目で判定が優である学生の学籍番号, 氏名, 系を問い合わせるための SQL 文を実行し, 実行結果を示せ。ただし, 条件文は「80 点以上」とせず, 「判定が優である」とすること。
- 3) 担当が「高田」または「三浦」の科目を履修している学生について, 学生名, 科目名, 担当, 判定を問い合わせる SQL 文を実行し, 実行結果を示せ。ただし, 表示するレコードは「高田」または「三浦」が担当している科目のもののみでよい。
- 4) 科目名に「学」が含まれる科目について, 履修している学生名, 科目名, 点数を問い合わせる SQL 文を実行し, 実行結果を示せ。
- 5) 担当が「三浦」の科目の平均点を問い合わせる SQL 文を実行し, 実行結果を示せ。

10. レポートに書く事項

レポートに含めてほしい内容を示す。なお、以下に示した項目の名称は、自分で適切な見出しを考えて章立てすること。テキストの説明をそのまま転載することは厳禁である。

- 1) 表紙
タイトル, 所属, 学籍番号, 組番号, 名前, 実験日を必ず記入すること。
- 2) 目的
この実験の目的を最初に記載すること。
- 3) 原理
SQL とは何か, データベースとは何かについて記載すること。
- 4) 実験環境
ハードウェア (PC スペック), ソフトウェア (バージョンなど) について記載する。
- 5) 課題
どのような課題で, 何をしたのかを記載する。課題に応じて, 表や画像, SQL 文, 実行結果を記載する。
- 6) 考察
今回の実験によって得られた考察を記載する。

11. 評価ポイント

期限内提出: 10%

実験実施態度と完了報告: 20%

レポートの内容: 70%

レポートの内容: 100 点満点

- 1) レポート全般: 30 点
 - レポートの体裁 (10)
 - 表紙, 章立てができているか, 誤字脱字はないか
 - SQL, データベースについて十分説明している (10)
 - 考察が十分書けている (10)
- 2) 課題 1: 30 点
 - 第 1 正規化: 10 点
 - 第 2 正規化: 10 点
 - 第 3 正規化: 10 点
- 3) 課題 2: 15 点
 - データベース・テーブルを生成する SQL 文: 5 点
 - 各テーブルにデータを挿入する SQL 文: 5 点
 - 各テーブルに登録した内容を表示する SQL 文とその実行結果: 5 点
- 4) 課題 3: 25 点 (5 点 x 5 問)
 - 1)~5)を実現する SQL 文とその実行結果: 各 5 点

12. 提出方法

以下のファイルを Teams の「課題」へ提出する.

レポート本体 (PDF ファイル)

- ファイル名: 組番号_氏名.pdf (例: 493_萩原隆義.pdf)
- レポートには表紙をつけること

SQL 文の書かれたファイル (sql ファイル)

- ファイル名: 組番号_氏名.sql (例: 493_萩原隆義.sql)
- .sql がうまく作成できない場合はテキストファイルでも可 (例: 493_萩原隆義.txt)
- データベースの作成、テーブルの作成、データの挿入、課題 1～3 が 1 つのファイルに含まれていること

参考文献

- [1] 株式会社ソフトエージェンシー, MySQL, Qooker, VG-Sync, IP-PBX | ライセンス早分かり
<http://www.softagency.co.jp/products/mysql/license>
- [2] 鈴木 啓修, MySQL 全機能リファレンス, 技術評論社, 2004
- [3] MySQL 5.6 リファレンスマニュアル, <https://dev.mysql.com/doc/refman/5.6/ja/>